

Towards a Universal Sequence Embedding: A Multi-Domain, Multi-Task Benchmark for Self-Supervised Methods

Egor Surkov
egorsurkov.ds@gmail.com
Applied AI
Moscow, Russia

Andrey Savochkin
andrey.20041@mail.ru
Russian State Medical University
Moscow, Russia

Grigorii Gulii
grigguliy@gmail.com
Applied AI
Moscow, Russia

Konstantin Sozykin
k.sozykin@applied-ai.ru
Applied AI
Moscow, Russia

Dmitry Osin
dima.tina2013@gmail.com
Applied AI
Moscow, Russia

Albert Bogdanov
albertbogdahov@gmail.com
Applied AI
Moscow, Russia

Aleksei Shestov
shestovmsu@gmail.com
SB AI Lab
Moscow, Russia

Artem Sakhno
sakhno.ad18@physics.msu.ru
SB AI Lab
Moscow, Russia

Maksim Makarenko
maksim.makarenko.research@gmail.com
SB AI Lab
Moscow, Russia

Andrey Savchenko
avsavchenko@hse.ru
SB AI Lab
Moscow, Russia

Evgeny Burnaev
e.burnaev@applied-ai.ru
Applied AI
Moscow, Russia

Egor Shvetsov
geksut@gmail.com
Applied AI
Moscow, Russia

Abstract

Self-supervised learning (SSL) has become the standard approach for producing reusable embeddings, with one practical advantage pretrain once: deploy across many downstream tasks. This reuse rests on an implicit assumption that an embedding performing well on one task will perform well on others yet SSL objectives are developed and validated against a single downstream task. This mismatch is especially acute for sequential data, where the temporal structure naturally gives rise to multiple downstream tasks and where multi-task reuse is the norm in areas such as finance and recommender systems. In this work, we want to test up to which degree this assumption holds. To test it, we collect a multi-domain benchmark containing 12 datasets from four structurally distinct families of sequential data (transactions, recommender logs, time series, text) and construct classification, regression, forecasting, and anomaly detection downstream targets, yielding 42 dataset-task settings. We evaluate self-supervised methods spanning contrastive, masked, latent-predictive, and autoregressive objectives under a single protocol, alongside a classical feature baseline and a randomly initialised encoder. We find that no method is universal and the strongest method leads in fewer than a quarter of the settings, almost every method wins somewhere, and even the untrained encoders remain competitive on several tasks. Rankings further

shift with the downstream validator and the embedding aggregation rule, so the evaluation protocol itself shapes conclusions. Fusing the embeddings of task-specialised encoders recovers the upper envelope of their performance. Overall, our results offer practical guidance for building and selecting sequence embeddings that serve many downstream tasks at once.

CCS Concepts

• Computing methodologies → Self-Supervised learning.

Keywords

Sequence embeddings, self-supervised learning, event sequences, multi-task evaluation, multi-domain benchmark, model selection, embedding fusion

ACM Reference Format:

Egor Surkov, Andrey Savochkin, Grigorii Gulii, Konstantin Sozykin, Dmitry Osin, Albert Bogdanov, Aleksei Shestov, Artem Sakhno, Maksim Makarenko, Andrey Savchenko, Evgeny Burnaev, and Egor Shvetsov. 2026. Towards a Universal Sequence Embedding: A Multi-Domain, Multi-Task Benchmark for Self-Supervised Methods. In . ACM, New York, NY, USA, 32 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 Introduction

Many practical systems would benefit from a *universal* sequence embedding, a single representation of a sequence that can be reused across several downstream tasks, such as for risk scoring, future-action forecasting, anomaly detection, and ranking. Despite the widespread deployment of sequence embeddings in finance, retail, and recommender systems [6, 30, 39, 44, 46], the community lacks a systematic benchmark for evaluating their generalization across domains and tasks. Self-supervised learning (SSL) has emerged as

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference'17, Washington, DC, USA

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
<https://doi.org/10.1145/nnnnnnn.nnnnnnn>

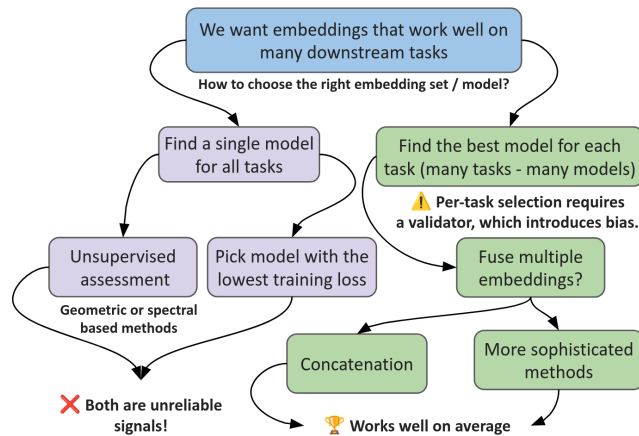


Figure 1: Possible strategies for obtaining embeddings that work well across many downstream tasks. Selecting a single model for all tasks requires a reliable label-free signal, but both the SSL training loss and unsupervised assessment turn out to be unreliable. Selecting the best model per task requires a downstream validator, which introduces its own bias. Fusing multiple embeddings sidesteps both issues and performs well on average.

a natural approach for learning such embedders, with recent methods targeting transactions, recommender logs, time series, and text-like sequences [4, 14, 18, 23, 35, 42, 45, 59]. Yet these methods are typically evaluated in isolation (on one or a few classification tasks within their home domain) leaving a fundamental question unanswered: can any SSL objective produce embeddings that work well across diverse tasks and domains?

This paper introduces a benchmark designed specifically to answer this question, assembling 12 datasets across four domains and constructing up to four downstream tasks per dataset under a unified evaluation protocol. Every target in a downstream task is annotated as *global* (a property of the whole sequence) or *local* (a property of a bounded future window). We publicly release the resulting benchmark. We then adapt seven SSL methods and complement them with three baselines: TSFresh [9] and two randomly initialised encoders. The pretrained encoders are evaluated under a unified protocol with extensive hyperparameter optimisation. The dataset and benchmark implementation are publicly available¹ under a permissive license for research use, with maintenance and issue tracking on the linked repository.

Unlike prior benchmarks such as MTEB [30] and SUPERB [58] which focus on a single domain (text or speech), or PEER [56] and BEACON [40] which target specific biological sequences, our benchmark spans four structurally distinct domains and four task families. This cross-domain, multi-task design captures the reality of industrial embedding systems, where a single representation

must serve ranking, forecasting, anomaly detection, and classification simultaneously. The benchmark’s explicit focus on evaluating frozen representations (the predominant deployment paradigm) distinguishes it from leaderboards that permit fine-tuning or evaluate only within-domain transfer [18, 23].

Our main contributions are (1) a constructed multi-domain, multi-task benchmark of 12 datasets spanning four domains and 42 dataset-task settings, (2) a unified comparison of seven SSL methods across different sequential domains (3) a study of the evaluation protocol itself, covering the downstream validator, the embedding aggregation rule, and the hyperparameter-selection regime, (4) an assessment of label-free model selection in the multi-task regime, and (5) an analysis of embedding fusion. Together, these lead us to the following findings:

(1) *Methods are not universal but there are some patterns.* The theory predicts that downstream success depends on the alignment between the relation used during pretraining and the target [5, 21]. While CoLES [4] has the best average rank (2.69), Slice-InfoNCE overtakes it on classification (2.55 vs. 2.73), and seven distinct methods win at least one of the 42 settings. Global² versus temporal³ tasks present the biggest change in method rankings. TSFresh average rank is 5 on temporal tasks and 7.5 on global ones. The next-token prediction objective is even more illustrative, NTP-GPT ranks third on classification but first on forecasting among all datasets. Since TSFresh is a purely statistical descriptor of inter-arrival times, we analyse in Appendix C.6 which of its temporal and spectral feature families carry this signal as an opportunity to improve existing methods.

(2) *The aggregation rule is selected by the task, not by the method.* Every encoder produces a sequence of hidden states and reduces them to one vector by mean-pooling or by taking the last state. We tune this aggregation rule for every dataset-task-method combination and find temporal-global connection. The single last state is chosen in 85% of forecasting and 60% of regression configurations, on a contrary mean-pooling is selected in 62% of classification and anomaly tasks.

(3) *Model and hyperparameter selection for a multi-task embedder is ill-posed.* Adapting a method to a new task or domain requires hyperparameter optimisation, which we find improves downstream scores up to 20% in our evaluation (Appendix Figure 10). Any such tuning requires an objective, however, no single validation signal corresponds to all downstream tasks at once. Comparing single- and multi-objective optimisation based on the Tree-structured Parzen Estimator, we find that the multi-objective variant significantly reduces computational cost at the price of slightly lower performance. Per-task selection, in turn, requires a downstream validator, and we demonstrate that the validator itself carries a bias, since different probes can rank the same embeddings differently. For example, MLP favours forecasting tasks while LightGBM favours classification. The overall procedure is outlined in Figure 1.

(4) *The natural label-free proxies do not rescue selection.* The SSL validation loss is the default signal for early stopping and checkpoint selection, yet it is already known to be uninformative about representation quality even in the single-task case [17]. This is

¹Dataset: <https://huggingface.co/datasets/On-Point-Rnd/ESdB-Embeddings-for-Sequential-data-Benchmark>; code: <https://github.com/On-Point-RND/ESdB-Embeddings-for-Sequential-data-Benchmark>

²Classification and anomaly detection.

³Forecasting and regression.

Table 1: Mean rank by downstream task family over the eight non-random methods (lower is better; R = regression, C = classification, F = forecasting, A = anomaly detection). Ranks account for variation across repeated downstream runs: methods with overlapping mean-performance intervals share a rank group. The best value per column is in bold and the second best is underlined.

Method	R	C	F	A	Overall
CoLES	2.10	2.73	3.73	2.10	2.69
Slice-InfoNCE	3.15	2.55	4.77	<u>3.20</u>	<u>3.43</u>
NTP-GRU	3.80	4.00	4.14	4.40	4.08
NTP-GPT	5.20	3.95	3.64	5.25	4.48
Mask-InfoNCE	4.75	4.45	5.18	3.95	4.60
JEPA	5.70	5.09	4.14	4.05	4.74
MLM	6.30	5.86	5.32	5.50	5.74
TSFresh	5.00	7.36	5.09	7.55	6.25

what motivated unsupervised representation-quality metrics [17, 27, 41, 47, 57] as label-free alternatives, moreover, such a criterion would let one encoder be tuned once rather than separately for every downstream task, which otherwise multiplies cost [36]. We evaluate the loss together with three such measures, RankMe [17], LiDAR [47], and aSMI [27], find that none of them reliably tracks downstream performance in the multi-task setting, and characterise their failure modes.

(5) *Since no single model suffices, one can train several and combine them.* This, however, only shifts the question to how the embeddings should be fused. We find that fusing the embeddings of two task-best methods can recover the best of both across tasks at once, yet the preferred fusion procedure depends on the validator, and naive concatenation is not uniformly the strongest choice. **We additionally propose** a Tucker decomposition-based fusion method that learns low-rank projection matrices from the data without explicitly building the full outer-product tensor, and test it against several linear and non-linear methods.

Organisation. §2 describes the source datasets, the construction of downstream targets, preprocessing, and the splitting protocols. §3 defines the evaluation protocol and §4 the pretraining objectives together with the statistical and random baselines. §5 establishes the findings above: the overall method comparison and cross-task agreement (§5.1), embedding aggregation (§5.2), the downstream validator (§5.3) together with single- versus multi-objective hyperparameter optimisation (§5.5), the limits of label-free selection (§5.4), and embedding fusion (§5.6). Appendix A reviews sequence SSL objectives, benchmarks for representation learning, and current model-selection practice. Appendix B gives the complete target matrix, class balances, and horizons for all 12 datasets. Appendix C provides details on SSL methods and contains the TS-Fresh feature-family analysis. Appendix D reports the search spaces and compute budgets. The derivation of the Tucker-based fusion procedure is presented in Appendix E.4 and the full fusion and single- versus multi-target results. The remaining part details the splitting protocol (Appendix F), the downstream validators (Appendix G), and the unsupervised selection metrics (Appendix H).

2 Datasets and Downstream Task

We construct the benchmark from 12 source datasets spanning four domains: transactions, recommender-system logs, time series, and text (Table 7). These domains were selected to cover the range of feature types (categorical, numerical, and mixed) and cardinalities encountered in real-world deployment and to represent distinct temporal structures: irregular event streams (transactions), interaction sequences (RecSys), regularly sampled measurements (time series), and token sequences (text). Datasets were chosen based on public availability, established use in prior SSL research, and sufficient sequence length to support multiple downstream tasks. We standardise them under a common interface in which each example is an ordered sequence associated with an entity.

2.1 Downstream Tasks

We consider four downstream task families: classification, regression, forecasting, and anomaly detection. Target scope is independent of task family: classification, regression, and anomaly detection may use either global or local targets, while forecasting targets are local. Across the 42 available dataset-task settings, we construct 35 targets from sequence histories, metadata, or training-set statistics and retain seven compatible source-provided targets as defaults. Appendix Tables 7 and 8 report the complete target coverage, scopes, and horizons. Classification targets represent categorical outcomes, regression targets quantify future value or activity over a fixed horizon, forecasting targets predict activity in the next time interval, and anomaly targets identify rare or tail behaviours. For some datasets, the available data do not support all four task families; we omit these tasks and mark them with $\times$ in Appendix Table 7.

2.2 Data Preprocessing

Each source dataset is converted into a common entity-centric representation, with categorical features encoded by frequency rank and numerical features normalised via robust dataset-specific scaling. For local targets, we sample multiple valid temporal cutoffs, or *shifts*, from each sequence. Every shift defines a history prefix and its horizon-specific future target, producing multiple downstream examples from a single long history without duplicating the underlying events. This is particularly important for datasets with few entities but long sequences, as reflected in Appendix Table 11. Full preprocessing and storage details are provided in Appendix B.2.

2.3 Splitting Protocol

For local targets, we follow the Global Time Split of Time to Split [20]. A single boundary T_{test} , defined as the global 0.9 quantile of valid target times, separates training and validation events from future test events across all entities. Target horizons determine which cutoffs are valid and are reported in Appendix Table 8. Because Alpha-Battle does not provide a shared absolute timeline, we preserve its source-provided train and test partitions.

For global targets, we follow the Global User Split [20] and assign complete entity histories to disjoint training and test sets in an approximately 90/10 proportion. Figure 2 summarises both protocols; implementation details and processed sample counts are provided in Appendix F and Table 11.

3 Evaluation Methodology

3.1 Downstream Evaluation

The final benchmark comparison uses a linear probe, a standard protocol for evaluating frozen SSL representations across vision, event sequences, and time series [3, 8, 18, 35]. In our implementation, it is a single linear layer trained on a GPU, which makes repeated evaluation during hyperparameter optimisation inexpensive. To study validator sensitivity, we additionally evaluate the same candidate embeddings with an MLP and LightGBM on six datasets. The MLP provides a nonlinear neural readout, while LightGBM provides a strong tree-based validator. For the final comparison, each pretrained encoder produces frozen embeddings once, after which the linear probe is trained with three random seeds. We report the mean and standard deviation over these probe runs (not repeated SSL pretraining); the same procedure is used for the fusion experiments. The uncertainty intervals used to form rank groups are the reported mean plus or minus one standard deviation and therefore quantify probe variability rather than repeated SSL pretraining.

3.2 Hyperparameter Optimisation

For each dataset and pretraining method, we run one multi-target study with Optuna [1] and its multivariate Tree-structured Parzen Estimator on a fixed data subsample. Each trial first trains an encoder using only its SSL objective. Once training terminates, we freeze the encoder, extract embeddings, and evaluate them once with a linear probe on every available downstream task; the resulting task metrics are returned jointly to Optuna. Downstream labels therefore guide selection among completed training runs but do not affect encoder gradients or within-run stopping. We select one hyperparameter configuration per task according to its validation metric, then retrain the encoder from scratch on the full training data using the selected hyperparameters and perform the final downstream evaluation. A post-hoc convergence analysis in

Appendix Figure 10 shows that downstream scores can continue to improve substantially beyond the initial random trials. Method-specific search spaces, stopping rules, and compute budgets are reported in Appendix D.

3.3 Model Selection Analyses

The HPO candidates support three model-selection analyses. We test whether the linear probe, MLP, and LightGBM select the same configurations; whether SSL loss can select across configurations and RankMe [17], LiDAR [47], and aSMI [27] can select checkpoints; and how multi-target HPO compares with separate single-target studies.

4 Pretraining Methods and Baselines

We implement the methods and baselines summarised in Table 2. All neural methods are adapted to the common sequence interface of Section 2.2; objective-specific heads are used only during pretraining, while downstream evaluation operates on frozen sequence embeddings. Detailed losses, architectures, augmentations, and embedding extraction rules are provided in Appendix C, with hyperparameter search spaces reported in Appendix D.

5 Experiments

5.1 Overall Method Comparison

Table 1 presents our main ranking result. For every dataset-task pair, methods with overlapping mean-performance intervals are assigned to the same rank group, and the group receives the mid-rank of the positions it spans. We then average these ranks over the datasets available for each task family. The per-dataset aggregation and complete results are provided in Appendix E.1.

No universal method. Based on mean downstream performance, seven of the eight non-random methods are the top method in at least one of the 42 dataset-task settings. Slice-InfoNCE records the most outright wins (14), while CoLES wins 10 settings but has the best average rank (2.69; Table 1). The task-specific profiles in Figure 4 confirm that CoLES is the strongest overall but not universally best. Forecasting is the clearest case: eleven datasets produce six distinct winners, no method wins more than three, and the average ranks of all pretrained methods are compressed into the narrowest range among the four tasks (Figure 12). Non-pretrained baselines are also competitive in individual settings: on 30Music regression, TSFresh gains 0.150 in R^2 over the stronger random encoder, compared with 0.039 for CoLES, while the untrained transformer is the best method on Zvuk regression. An embedding that is best for every task and dataset is therefore not available among current SSL objectives.

Downstream tasks are weakly aligned. Figure 5 compares downstream tasks using both standardised method scores and within-dataset method rankings. The score-based relationships are weak throughout: classification and anomaly detection have the largest shared variance ($R^2 = 0.26$), regression and forecasting share only 0.16, and classification provides essentially no predictive signal for forecasting ($R^2 < 0.01$). The rank-based analysis gives the same conclusion, with mean Kendall agreement of only $\tau_b = 0.11$ –0.32 across task pairs. Classification and anomaly detection agree the

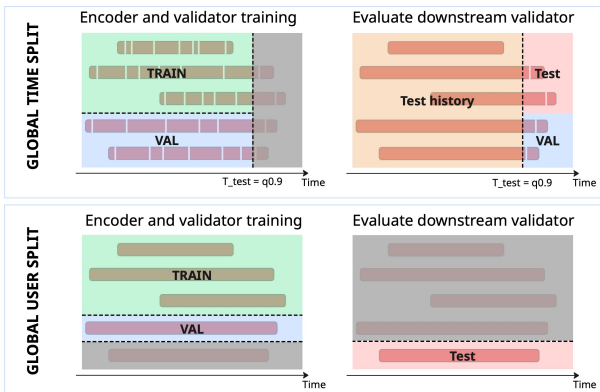
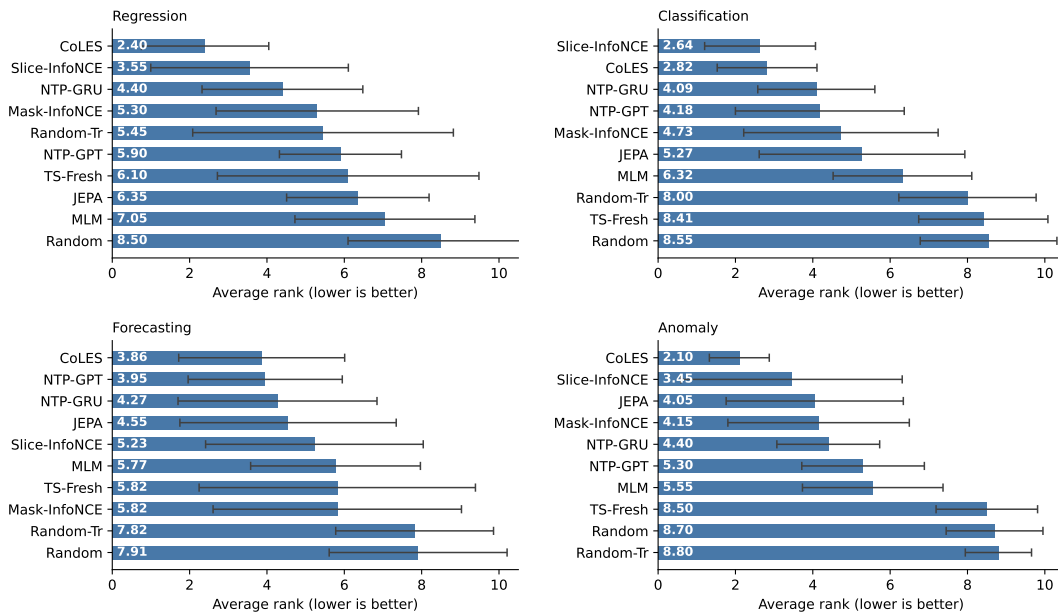


Figure 2: Data splitting protocols. The global time split reserves events after T_{test} for temporal testing, while histories used to train embeddings and downstream validators remain before the boundary. The global user split partitions complete entity histories between training and test sets.

Table 2: Implemented objective families and representative areas from which they are inherited or in which they have previously been applied.

Family	Implemented models	Origins and prior applications
Contrastive learning	CoLES; Slice-InfoNCE; Mask-InfoNCE	Event sequences/transactions: CoLES [4], MLEM [29]. RecSys: CL4SRec [54]. Vision: SimCLR [8]. Text: SimCSE [16], E5 [50].
Masked modelling	MLM	Text: BERT [12]. RecSys: BERT4Rec [45], S ³ -Rec [62]. Event sequences/transactions: PRAGMA [35]. Time series: PatchTST [32], SimMTM [13], TimesBERT [60]. Vision: SimMIM [55].
Latent prediction	JEPA	Vision: I-JEPA [3]. RecSys: JEPA4Rec [31]. Text/LLM: LLM-JEPA [24].
Next-token prediction	NTP-GPT; NTP-GRU	Text: GPT-2 [38]. RecSys: SASRec [26]. Transactions: TransactionGPT [14]. Time series: Chronos [2].
Statistical baseline	TSFresh	Time series and event sequences: TSFresh [9].
Random baselines	R-RNN; R-Tr	Untrained GRU and GPT-style Transformer sequence encoders.

**Figure 3: Average rank of evaluated methods across datasets per four tasks, lower is better.**

most, while classification and forecasting agree the least. Thus, success on one downstream task does not reliably transfer to another, even within the same dataset.

Objectives transfer across domains. Our results argue against restricting each domain to domain-specific objectives. Cross-domain adaptations repeatedly lead outside their source settings (Tables 14–17): vision-derived Slice-InfoNCE outperforms transaction-specific CoLES on classification in two of three transaction datasets (AGE, +0.210; Retail, +0.132 over the stronger random encoder); JEPA leads demand forecasting on Rossmann, Taobao, and Favorita and anomaly detection on AGE; and language-modeling-based NTP-GRU leads event-count forecasting on AGE, AlphaBattle, and Electric Devices. Transfer remains uneven: on Favorita regression all pretrained methods underperform the random RNN, while on ETT forecasting only the two InfoNCE variants improve over a constant

predictor. Objectives from vision and language modelling should therefore be evaluated alongside domain-specific methods.

5.2 Embedding Aggregation Selection

Each encoder produces a sequence of hidden states, from which Optuna selects either mean-pooling or the last state for each dataset–task–method combination. Figure 6 shows that forecasting selects the last state in 85% of configurations and regression in 60%, whereas classification and anomaly detection select mean-pooling in 62% of configurations. This pattern aligns with target scope: the last state is selected in 68% of local-target configurations but only 36% of global-target configurations, with the same direction for every method analysed. Local targets depend on behaviour near a temporal cutoff, whereas global targets describe the entire sequence. Fixing a single aggregation scheme can therefore favour one target family. Method- and dataset-level results are provided in Appendix E.2.

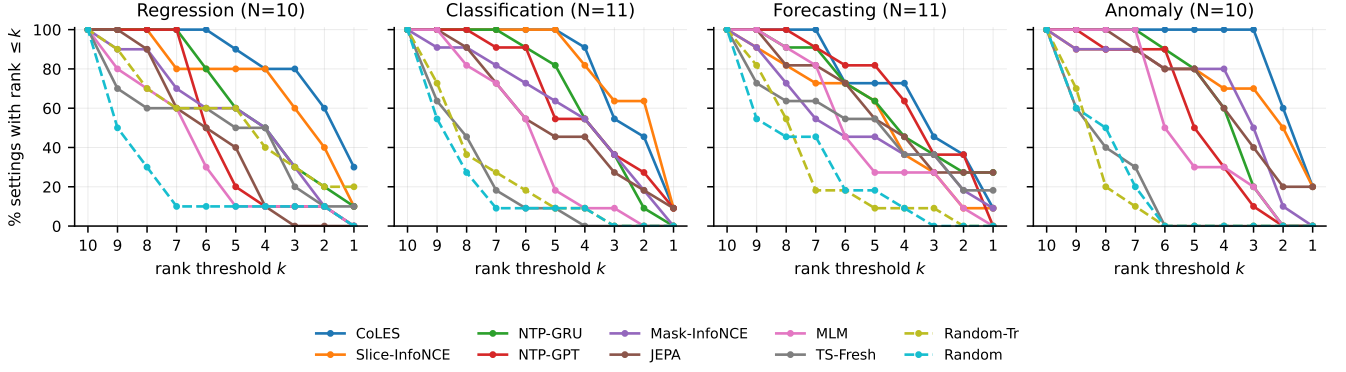


Figure 4: Task-specific uncertainty-aware rank profiles over all ten methods: within each task family, the curves show the share of dataset settings in which a method’s mid-rank is at most k (higher is better). The horizontal axis is reversed so that the strictest threshold ($k = 1$) is on the right; dashed curves denote the two random encoders. Ranks use the same mean-performance intervals and tie groups as Table 13.

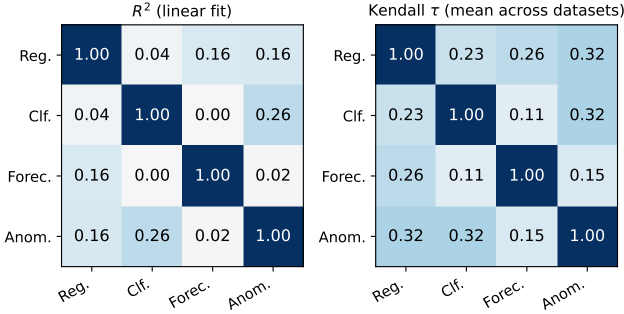


Figure 5: Cross-task relationship of downstream performance. Left: R^2 from a univariate linear fit between each pair of task columns after method scores are standardised within every dataset-task pair; observations are pooled dataset-method combinations. Right: Kendall’s τ_b between method rankings, computed separately within every dataset and averaged across datasets. Both analyses use the eight non-random methods. Low off-diagonal values show that performance and method ordering transfer only weakly between downstream tasks.

5.3 Downstream Validator Disagreement

To assess validator dependence, we rank the same candidate embeddings with each of the three validators within 23 available dataset-task settings from six datasets and aggregate the ranks overall and by task family; lower values are better.

Table 3 shows that the downstream validator can change the apparent ranking of embeddings even near the end of hyperparameter optimisation. LightGBM has the best overall average rank, but the task-level ordering is not uniform: MLP is strongest on forecasting tasks, while the linear probe is strongest on anomaly targets. Thus, the choice of validator is not merely a reporting detail; it

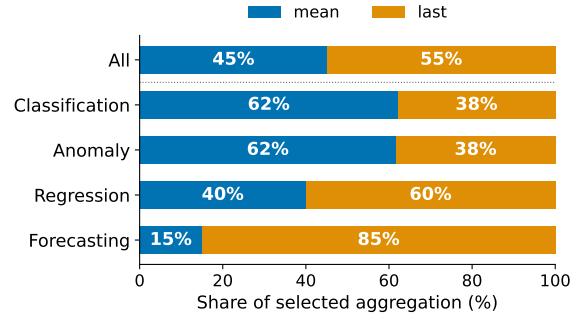


Figure 6: Share of embedding aggregations (mean-pooling vs. last hidden state) selected by hyperparameter optimisation, per task family and overall. Each row aggregates all dataset-method combinations of the task.

Table 3: Validator sensitivity over 23 settings from six datasets on the final 10 Optuna trials. Values are average ranks of the candidates selected by each validator; lower is better.

Statistic	Scope	LightGBM	MLP	Linear probe
Rank	Avg.	1.652	2.000	2.043
Rank	Cls.	1.167	2.167	2.500
Rank	Forecast	2.000	1.400	1.800
Rank	Reg.	1.333	2.333	2.333
Rank	Anomaly	2.167	2.000	1.500

can determine which encoder or hyperparameter configuration appears preferable. Full first-versus-final-trial comparisons and pairwise validator correlations are reported in Appendix E.3.

Table 4: Task-specific configuration trade-offs within one multi-target HPO study on X5-Retail. Each row is one fixed encoder configuration selected by the validation metric of the indicated task and then evaluated on all four tasks. Higher is better, the best value within each encoder block is in bold. Per task single (S) HPO requires $\times 3 - 4$ times more compute on average.

Encoder	Selected for	Reg.	Clf.	Forec.	Anom.
NTP-GRU	Regression	0.118	0.339	0.348	0.640
	Classification	0.103	0.443	0.133	0.658
	Forecasting	0.121	0.426	0.428	0.648
	Anomaly	0.076	0.407	0.140	0.663
JEPA	Regression	0.146	0.355	0.272	0.660
	Classification	0.063	0.434	0.185	0.591
	Forecasting	0.091	0.441	0.336	0.657
	Anomaly	0.041	0.427	0.132	0.692

5.4 The Limits of Label-Free Model Selection

The disagreement above still assumes access to labelled downstream tasks. We therefore test two label-free alternatives at different stages of model selection: the SSL loss for selecting among hyperparameter configurations, and unsupervised representation metrics for selecting a checkpoint within a training run.

Selection among hyperparameter configurations. For CoLES and NTP-GPT on three datasets from distinct domains (AGE, Twitter, and Yambda), we compute the Spearman correlation between the SSL loss and every downstream metric across HPO trials. Figure 8 shows no consistent relationship: correlations span from -1 to $+1$, with means close to zero for every downstream task under both training and validation loss. SSL loss therefore does not provide a reliable configuration-selection signal.

Checkpoint selection within a training run. We next track RankMe [17], LiDAR [47], and aSMI [27] at every checkpoint. For NTP-GPT on AGE regression, the metric variants peak at different epochs and do not consistently coincide with the best downstream checkpoint (Figure 7). Additional tasks, encoders, and datasets show the same ambiguity in Appendix H. Thus, neither SSL loss nor the tested unsupervised metrics provides an unambiguous label-free selection rule in the evaluated settings.

5.5 Task-Specific Configuration Selection

Multi-target HPO evaluates every candidate configuration on every downstream task, allowing us to test whether one fixed configuration can serve all tasks. Table 4 reports configurations selected from the same joint Optuna study for two encoders on X5-Retail. Each row is one fixed configuration evaluated on all four tasks. No row is best throughout: for JEPA, the regression-selected configuration obtains the best regression score but loses 0.086 classification accuracy to the forecasting-selected configuration; the anomaly-selected configuration is best on anomaly detection but has the weakest regression score. NTP-GRU exhibits the same task-specific trade-off. Even within a fixed encoder and dataset, downstream tasks therefore select different embeddings.

Single- versus multi-target HPO. Single-target HPO runs one 100-trial Optuna study per downstream task, requiring 400 trials on X5-Retail and 300 on Twitter. Multi-target HPO jointly optimises

Table 5: Best downstream metric attained by single-target (S) and multi-target (M) HPO on each task (higher is better). The better result per dataset–encoder–task is in bold.

Dataset	Model	HPO	Reg.	Clf.	Forec.	Anom.
X5-Retail	NTP-GRU	S	0.160	0.454	0.431	0.683
		M	0.121	0.443	0.428	0.663
X5-Retail	JEPA	S	0.125	0.463	0.325	0.691
		M	0.146	0.441	0.336	0.692
Twitter	NTP-GRU	S	0.685	0.744	–	0.997
		M	0.677	0.751	–	0.997
Twitter	JEPA	S	0.450	0.647	–	0.998
		M	0.480	0.646	–	0.994

all available downstream metrics in one 100-trial study, reducing the trial count fourfold and threefold, respectively. Table 5 compares the two regimes on two datasets and two encoders. Multi-target HPO improves some dataset–encoder–task settings and degrades others; neither regime is uniformly preferable. It provides a cheaper search for task-specific configurations, while additional single-target tuning remains useful when compute permits. Full results are given in Appendix E.6.

5.6 Embedding Fusion

To illustrate a practical use case enabled by our benchmark (combining task-specialized representations), we study embedding fusion on AGE, which covers all four task families. For each task we take the method ranked best on it in Subsection 5.1, form pairs of the resulting encoders, and fuse the two frozen embeddings using concatenation, PCA [43], CCA [7, 22], KrossFuse [52], and the Tucker-decomposition-based TuckerFactorConcat [49] (see Appendix E.4). This demonstrates how the benchmark can evaluate composite embedding strategies beyond single-method comparisons.

Table 6 shows the pair Slice-InfoNCE (best on classification) and NTP-GRU (best on forecasting) under the linear probe. The two parents have complementary profiles, and the fused embedding recovers close to the upper envelope of both on every task simultaneously: TuckerFactorConcat is the best variant on two of the four tasks. The choice of fusion procedure matters, and the best procedure depends on the validator: under LightGBM, plain concatenation is the strongest variant for this pair (Appendix E.5), consistent with the validator sensitivity discussed above. Appendix E.5 reports the remaining AGE pairs and repeats the analysis on Rossmann, showing that combining task-specialised embeddings can improve downstream performance on both datasets.

6 Discussion

Taken together, the selection results make the intended downstream task portfolio part of the model specification: no tested label-free signal removes this dependency. When labelled validation data are available, they should therefore be used to select completed pre-training runs. Multi-target HPO amortises this evaluation across tasks while retaining task-specific configuration choices.

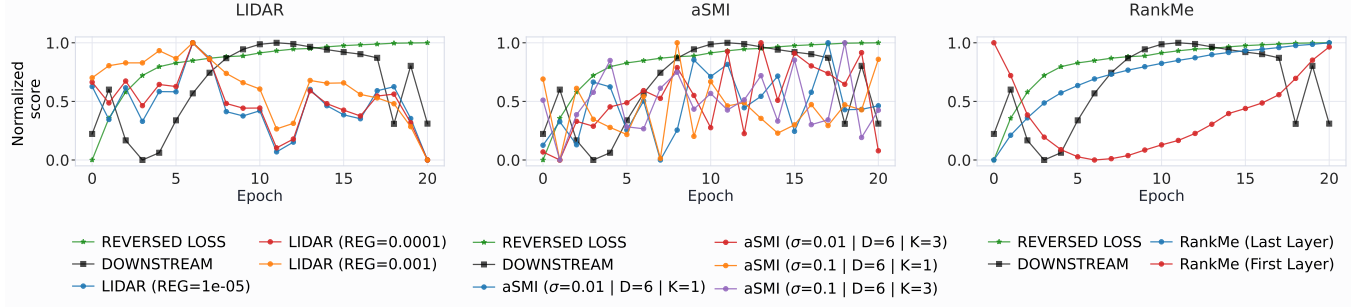


Figure 7: Checkpoint selection with unsupervised representation metrics for NTP-GPT trained on AGE regression. Curves are normalised for visual comparison. LiDAR is shown for several covariance regularisation values (REG); aSMI varies its Gaussian-noise scale σ , projection dimension D , and number of neighbours K ; RankMe is computed on the final embedding and immediately after input-feature encoding (*first layer*). The variants peak at different epochs and do not consistently select the downstream-best checkpoint.

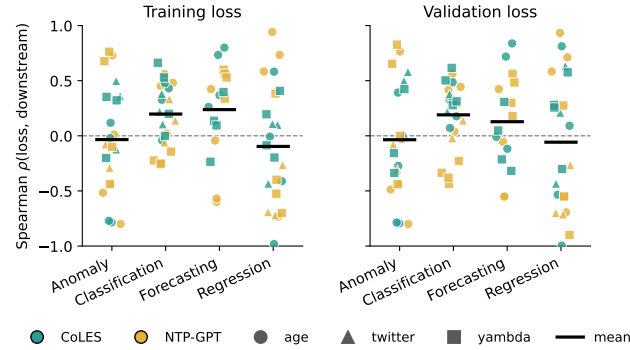


Figure 8: Spearman correlation between the SSL loss and each downstream metric across HPO trials for CoLES and NTP-GPT on AGE, Twitter, and Yambda. Each point is one encoder–dataset–pretraining–task combination; the black bar is the per-task mean. Negative values indicate that lower SSL loss is associated with better downstream performance.

Table 6: Embedding fusion on AGE (linear probe): the two source methods (top) and fused variants (bottom). Values are mean $\pm$ standard deviation across repeated downstream runs. The best mean per column is in bold, the second best is underlined.

	Classification accuracy	Regression R^2	Forecasting R^2	Anomaly ROC-AUC
Slice-InfoNCE (Classification)	0.630 $\pm$ 0.000	0.459 $\pm$ 0.002	0.244 $\pm$ 0.001	0.745 $\pm$ 0.002
NTP-GRU (Forecasting)	0.602 $\pm$ 0.003	0.491 $\pm$ 0.000	<u>0.411</u> $\pm$ 0.002	0.673 $\pm$ 0.006
Concatenation	0.630 $\pm$ 0.002	<u>0.498</u> $\pm$ 0.002	<u>0.411</u> $\pm$ 0.001	0.741 $\pm$ 0.006
PCA	0.627 $\pm$ 0.002	0.494 $\pm$ 0.001	0.404 $\pm$ 0.000	0.757 $\pm$ 0.010
CCA	0.620 $\pm$ 0.003	0.486 $\pm$ 0.000	0.321 $\pm$ 0.001	0.725 $\pm$ 0.005
KrossFuse	0.621 $\pm$ 0.003	0.454 $\pm$ 0.001	0.358 $\pm$ 0.001	0.685 $\pm$ 0.005
TuckerFactorConcat	<u>0.628</u> $\pm$ 0.001	0.499 $\pm$ 0.001	0.414 $\pm$ 0.001	<u>0.746</u> $\pm$ 0.007

Fusion addresses task specialisation but increases serving cost by requiring several encoders. A more practical direction is to train

one encoder with complementary SSL losses, for example combining contrastive, masked, latent-predictive, or autoregressive signals whose biases match the intended task portfolio. The loss weights and stopping point would still need downstream validation: our results provide no evidence that the combined pretraining loss alone would identify the best multi-task representation.

Limitations. The common interface trades some data scale for method comparability. We cap high-cardinality categorical vocabularies so that every evaluated method can use the same representation; consequently, the benchmark does not test modelling at the original industrial cardinalities. Supporting substantially larger vocabularies would require scalable categorical prediction heads and remains future work. The fusion study also covers only a small subset of the benchmark datasets.

7 Conclusion

We introduced a multi-domain benchmark for evaluating sequence embeddings across 42 dataset-task settings under a unified protocol. The benchmark provides three key contributions: (1) a curated collection of 12 datasets with standardized preprocessing and reproducible train/test splits; (2) a systematic evaluation protocol covering four task families with three validation signals; and (3) empirical findings that no SSL objective is universally best, that downstream task, validator, and embedding aggregation critically affect conclusions, and that objectives transferred from other domains are unexpectedly competitive. Multi-target HPO and embedding fusion emerge as practical strategies for exploiting task-specific strengths. The benchmark implementation, including preprocessing scripts, data loaders, model implementations, and evaluation harness, is released under a permissive license with versioned releases, comprehensive documentation, and an issue tracker. All processed datasets are provided in a unified Parquet format with reproducible train/test splits and seed-controlled randomizations. Experimental results are accompanied by hyperparameter configurations, random seeds, and hardware specifications to ensure full reproducibility of the reported findings.

References

- [1] Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. 2019. Optuna: A Next-generation Hyperparameter Optimization Framework. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*. 2623–2631. doi:10.1145/3292500.3330701
- [2] Abdul Fatir Ansari, Lorenzo Stella, Caner Turkmen, Xiyuan Zhang, Pedro Mercado, Huibin Shen, Oleksandr Shchur, Syama Sundar Rangapuram, Sebastian Pineda Arango, Shubham Kapoor, Danielle C. Maddix, Hao Wang, Michael W. Mahoney, Kari Torkkola, Andrew Gordon Wilson, Michael Bohlke-Schneider, and Yuyang Wang. 2024. Chronos: Learning the Language of Time Series. *arXiv preprint arXiv:2403.07815* (2024). <https://arxiv.org/abs/2403.07815>
- [3] Mahmoud Assran, Quentin Duval, Ishan Misra, Piotr Bojanowski, Pascal Vincent, Michael Rabbat, Yann LeCun, and Nicolas Ballas. 2023. Self-Supervised Learning from Images with a Joint-Embedding Predictive Architecture. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 15619–15629. <https://arxiv.org/abs/2301.08243>
- [4] Dmitrii Babaev, Nikita Ovsov, Ivan Kireev, Maria Ivanova, Gleb Gusev, Ivan Nazarov, and Alexander Tuzhilin. 2022. CoLES: Contrastive Learning for Event Sequences with Self-Supervision. In *Proceedings of the 2022 International Conference on Management of Data*. 1190–1199. doi:10.1145/3514221.3526129
- [5] Randall Balestriero and Yann LeCun. 2022. Contrastive and Non-Contrastive Self-Supervised Learning Recover Global and Local Spectral Embedding Methods. *arXiv preprint arXiv:2205.11508* (2022). <https://arxiv.org/abs/2205.11508>
- [6] Daniel Cer, Yinfei Yang, Sheng-yi Kong, Nan Hua, Nicole Limtiaco, Rhomni St. John, Noah Constant, Mario Guajardo-Cespedes, Steve Yuan, Chris Tar, Brian Strope, and Ray Kurzweil. 2018. Universal Sentence Encoder for English. In *Proceedings of EMNLP: System Demonstrations*.
- [7] James Chapman, Hao-Ting Wang, Lennie Wells, and Johannes Wiesner. 2021. CCA-Zoo: A collection of Regularized, Deep Learning based, Kernel, and Probabilistic CCA methods in a scikit-learn style framework. *Journal of Open Source Software* 6, 68 (2021), 3823. doi:10.21105/joss.03823
- [8] Ting Chen, Simon Kornblith, Mohammad Norouzi, and Geoffrey Hinton. 2020. A Simple Framework for Contrastive Learning of Visual Representations. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*. 1597–1607. <https://arxiv.org/abs/2002.05709>
- [9] Maximilian Christ, Nils Braun, Julius Neuffer, and Andreas W. Kempa-Liehr. 2018. Time Series Feature Extraction on Basis of Scalable Hypothesis Tests (tsfresh — A Python Package). *Neurocomputing* 307 (2018), 72–77. <https://doi.org/10.1016/j.neucom.2018.03.067>
- [10] Andrzej Cichocki, Namgil Lee, Ivan Oseledets, Anh-Huy Phan, Qibin Zhao, and Danilo P. Mandic. 2016. Tensor Networks for Dimensionality Reduction and Large-Scale Optimization Part 1 Low-Rank Tensor Decompositions. *Foundations and Trends in Machine Learning* 9, 4-5 (12 2016), 249–429. [arXiv:https://www.emerald.com/ftml/article-pdf/9/4-5/249/11154084/2200000059en.pdf](https://www.emerald.com/ftml/article-pdf/9/4-5/249/11154084/2200000059en.pdf) doi:10.1561/22000000059
- [11] Paul Covington, Jay Adams, and Emre Sargin. 2016. Deep Neural Networks for YouTube Recommendations. In *Proceedings of the 10th ACM Conference on Recommender Systems (RecSys)*. 191–198.
- [12] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. 4171–4186. <https://arxiv.org/abs/1810.04805>
- [13] Jiaxiang Dong, Haixu Wu, Haoran Zhang, Li Zhang, Jianmin Wang, and Ming-sheng Long. 2023. SimMTM: A Simple Pre-Training Framework for Masked Time-Series Modeling. In *Advances in Neural Information Processing Systems*. <https://arxiv.org/abs/2302.00861>
- [14] Yingdong Dou, Zhimeng Jiang, Tianyi Zhang, Mingzhi Hu, Zhichao Xu, Shubham Jain, Uday Singh Saini, Xiran Fan, Jiarui Sun, Menghai Pan, Junpeng Wang, Xin Dai, Liang Wang, Chin-Chia Michael Yeh, Yujie Fan, Vineeth Rakesh, Huiyuan Chen, Mangesh Bendre, Zhongfang Zhuang, Xiaoting Li, Prince Aboagye, Vivian Lai, Minghua Xu, Hao Yang, Yiwei Cai, Mahashweta Das, and Yuzhong Chen. 2025. TransactionGPT. *arXiv preprint arXiv:2511.08939* (2025). <https://arxiv.org/abs/2511.08939>
- [15] Linus Ericsson, Henry Gouk, and Timothy M. Hospedales. 2021. How Well Do Self-Supervised Models Transfer?. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 5414–5423.
- [16] Tianyu Gao, Xingcheng Yao, and Danqi Chen. 2021. SimCSE: Simple Contrastive Learning of Sentence Embeddings. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*. 6894–6910. <https://arxiv.org/abs/2104.08821>
- [17] Quentin Garrido, Randall Balestriero, Laurent Najman, and Yann LeCun. 2023. RankMe: Assessing the Downstream Performance of Pretrained Self-Supervised Representations by Their Rank. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*. 10929–10974. <https://arxiv.org/abs/2210.02885>
- [18] Mononito Goswami, Konrad Szafer, Arjun Choudhry, Yifu Cai, Shuo Li, and Artur Dubrawski. 2024. MOMENT: A Family of Open Time-Series Foundation Models. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*. <https://arxiv.org/abs/2402.03885>
- [19] Mihajlo Grbovic and Haibin Cheng. 2018. Real-time Personalization using Embeddings for Search Ranking at Airbnb. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*. 311–320.
- [20] Danil Gusak, Anna Volodkevich, Anton Klenitskiy, Alexey Vasilev, and Evgeny Frolov. 2025. Time to Split: Exploring Data Splitting Strategies for Offline Evaluation of Sequential Recommenders. *arXiv preprint arXiv:2507.16289* (2025). doi:10.1145/3705328.3748164
- [21] Jeff Z. HaoChen and Tengyu Ma. 2023. A Theoretical Study of Inductive Biases in Contrastive Learning. *arXiv preprint arXiv:2211.14699* (2023). <https://arxiv.org/abs/2211.14699>
- [22] David R. Hardoon, Sandor Szedmak, and John Shawe-Taylor. 2004. Canonical Correlation Analysis: An Overview with Application to Learning Methods. *Neural Computation* 16, 12 (12 2004), 2639–2664. [arXiv:https://direct.mit.edu/neco/article-pdf/16/12/2639/815992/0899766042321814.pdf](https://direct.mit.edu/neco/article-pdf/16/12/2639/815992/0899766042321814.pdf) doi:10.1162/0899766042321814
- [23] Yupeng Hou, Shanlei Mu, Wayne Xin Zhao, Yaliang Li, Bolin Ding, and Ji-Rong Wen. 2022. Towards Universal Sequence Representation Learning for Recommender Systems. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 585–593. doi:10.1145/3534678.3539381
- [24] Hai Huang, Yann LeCun, and Randall Balestriero. 2025. LLM-JEPA: Large Language Models Meet Joint Embedding Predictive Architectures. *arXiv preprint arXiv:2509.14252* (2025). <https://arxiv.org/abs/2509.14252>
- [25] Yangqin Jiang, Xubin Ren, Lianghao Xia, Da Luo, Kangyi Lin, and Chao Huang. 2025. RecGPT: A Foundation Model for Sequential Recommendation. *arXiv preprint arXiv:2506.06270* (2025). <https://arxiv.org/abs/2506.06270>
- [26] Wang-Cheng Kang and Julian McAuley. 2018. Self-Attentive Sequential Recommendation. In *2018 IEEE International Conference on Data Mining (ICDM)*. 197–206. <https://arxiv.org/abs/1808.09781>
- [27] Mikhail Kuznetsov, Ivan Butakov, Marina Munkhoeva, Alexey Frolov, and Ivan Oseledets. 2026. Information-Theoretic Unsupervised Embedding Quality Evaluation. *ICLR 2026 Workshop on Representational Alignment (Re-Align)* (2026). <https://openreview.net/forum?id=EwaNoLExT>
- [28] Zhe Li, Zhongwen Rao, Lujia Pan, Pengyun Wang, and Zenglin Xu. 2023. Ti-MAE: Self-Supervised Masked Time Series Autoencoders. *arXiv preprint arXiv:2301.08871* (2023). <https://arxiv.org/abs/2301.08871>
- [29] Viktor Moskvoretiskii, Dmitry Osin, Egor Shvetsov, Igor Udovichenko, Maxim Zheleznin, Andrey Dukhovny, Anna Zhimerikina, and Evgeny Burnaev. 2024. MLEM: Generative and Contrastive Learning as Distinct Modalities for Event Sequences. *arXiv preprint arXiv:2401.15935* (2024).
- [30] Niklas Muennighoff, Nouamane Tazi, Loïc Magne, and Nils Reimers. 2023. MTEB: Massive Text Embedding Benchmark. In *Proceedings of EACL*.
- [31] Minh-Anh Nguyen and Dung D. Le. 2025. JEPA4Rec: Learning Effective Language Representations for Sequential Recommendation via Joint Embedding Predictive Architecture. In *Proceedings of the Conference on Language Modeling (COLM)*. <https://arxiv.org/abs/2504.10512>
- [32] Yuqi Nie, Nam H. Nguyen, Phanwadee Sinthong, and Jayant Kalagnanam. 2023. A Time Series is Worth 64 Words: Long-term Forecasting with Transformers. In *International Conference on Learning Representations*. <https://arxiv.org/abs/2211.14730>
- [33] Ivan Oseledets. 2011. Tensor-train decomposition. *SIAM Journal on Scientific Computing* 33, 5 (2011).
- [34] Dmitry Osin, Igor Udovichenko, Egor Shvetsov, Viktor Moskvoretiskii, and Evgeny Burnaev. 2025. EBES: Easy Benchmarking for Event Sequences. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 5730–5741. <https://arxiv.org/abs/2410.03399>
- [35] Maxim Ostroukhov, Ruslan Mikhailov, Vladimir Iashin, Artem Sokolov, Andrei Akshonov, Vitaly Protasov, Dmitrii Beloborodov, Vince Mullin, Roman Yokunda Enzmann, Georgios Kolovos, Jason Renders, Pavel Nesterov, and Anton Repushko. 2026. PRAGMA: Revolut Foundation Model. *arXiv preprint arXiv:2604.08649* (2026). <https://arxiv.org/abs/2604.08649>
- [36] Yoshihiko Ozaki, Yuki Tanigaki, Shuhei Watanabe, and Masaki Onishi. 2020. Multiobjective Tree-structured Parzen Estimator for Computationally Expensive Optimization Problems. In *Proceedings of the 2020 Genetic and Evolutionary Computation Conference (GECCO)*. 533–541.
- [37] Aditya Pal, Chantat Eksombatchai, Yitong Zhou, Bo Zhao, Charles Rosenberg, and Jure Leskovec. 2020. PinnerSage: Multi-Modal User Embedding Framework for Recommendations at Pinterest. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*. 2311–2320.
- [38] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. 2019. Language Models are Unsupervised Multitask Learners. *OpenAI technical report* (2019). https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf

- [39] Nils Reimers and Iryna Gurevych. 2019. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In *Proceedings of EMNLP-IJCNLP*.
- [40] Yuchen Ren, Zhiyuan Chen, Lifeng Qiao, Hongtai Jing, and Others. 2024. BEA-CON: Benchmark for Comprehensive RNA Tasks and Language Models. In *Advances in Neural Information Processing Systems 37 (NeurIPS), Datasets and Benchmarks Track*.
- [41] Aleksei Shestov, Anton Klenitskiy, Daria Denisova, Amurkhan Dzagkoev, Daniil Petrovich, Andrey Savchenko, and Maksim Makarenko. 2026. Topological Metric for Unsupervised Embedding Quality Evaluation. In *European Conference on Information Retrieval*. Springer, 596–605.
- [42] Aleksei Shestov, Omar Zoloev, Maksim Makarenko, Mikhail Orlov, Egor Fadeev, Ivan Kireev, and Andrey Savchenko. 2025. LLM4ES: Learning user embeddings from event sequences via large language models. In *Proceedings of the 34th ACM International Conference on Information and Knowledge Management*. 5238–5242.
- [43] Jonathon Shlens. 2014. A Tutorial on Principal Component Analysis. arXiv:1404.1100 [cs.LG] <https://arxiv.org/abs/1404.1100>
- [44] Hongjin Su, Weijia Shi, Jungo Kasai, Yizhong Wang, Yushi Hu, Mari Ostendorf, Wen-tau Yih, Noah A. Smith, Luke Zettlemoyer, and Tao Yu. 2023. One Embedder, Any Task: Instruction-Finetuned Text Embeddings. In *Findings of ACL*.
- [45] Fei Sun, Jun Liu, Jian Wu, Changhua Pei, Xiao Lin, Wenwu Ou, and Peng Jiang. 2019. BERT4Rec: Sequential Recommendation with Bidirectional Encoder Representations from Transformer. In *Proceedings of the 28th ACM International Conference on Information and Knowledge Management (CIKM)*. 1441–1450. <https://arxiv.org/abs/1904.06690>
- [46] Egor Surkov, Dmitry Osin, Evgeny Burnaev, and Egor Shvetsov. 2025. How to model Human Actions distribution with Event Sequence Data. *arXiv preprint arXiv:2510.05856* (2025).
- [47] Vimal Thilak, Chen Huang, Omid Saremi, Laurent Dinh, Hanlin Goh, Preetum Nakkiran, Joshua M. Susskind, and Etai Littwin. 2023. LiDAR: Sensing Linear Probing Performance in Joint Embedding SSL Architectures. *arXiv preprint arXiv:2312.04000* (2023). <https://arxiv.org/abs/2312.04000>
- [48] Yonglong Tian, Chen Sun, Ben Poole, Dilip Krishnan, Cordelia Schmid, and Phillip Isola. 2020. What Makes for Good Views for Contrastive Learning?. In *Advances in Neural Information Processing Systems 33 (NeurIPS)*.
- [49] Ledyard R. Tucker. 1966. Some mathematical notes on three-mode factor analysis. *Psychometrika* 31, 3 (sep 1966), 279–311. doi:10.1007/BF02289464
- [50] Liang Wang, Nan Yang, Xiaolong Huang, Binxing Jiao, Linjun Yang, Daxin Jiang, Rangan Majumder, and Furu Wei. 2022. Text Embeddings by Weakly-Supervised Contrastive Pre-training. *arXiv preprint arXiv:2212.03533* (2022). <https://arxiv.org/abs/2212.03533>
- [51] Gerald Woo, Chenghao Liu, Akshat Kumar, Caiming Xiong, Silvio Savarese, and Doyen Sahoo. 2024. Unified Training of Universal Time Series Forecasting Transformers. *arXiv preprint arXiv:2402.02592* (2024). <https://arxiv.org/abs/2402.02592>
- [52] Youqi WU, Jingwei Zhang, and Farzan Farnia. 2025. When Kernels Multiply, Clusters Unify: Fusing Embeddings with the Kronecker Product. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*. <https://openreview.net/forum?id=XougXwZAHl>
- [53] Tete Xiao, Xiaolong Wang, Alexei A. Efros, and Trevor Darrell. 2021. What Should Not Be Contrastive in Contrastive Learning. In *International Conference on Learning Representations (ICLR)*.
- [54] Xu Xie, Fei Sun, Zhaoyang Liu, Shiwen Wu, Jinyang Gao, Bolin Ding, and Bin Cui. 2022. Contrastive Learning for Sequential Recommendation. In *Proceedings of the 38th IEEE International Conference on Data Engineering*. <https://arxiv.org/abs/2010.14395>
- [55] Zhenda Xie, Zheng Zhang, Yue Cao, Yutong Lin, Jianmin Bao, Zhuliang Yao, Qi Dai, and Han Hu. 2022. SimMIM: A Simple Framework for Masked Image Modeling. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 9653–9663. <https://arxiv.org/abs/2111.09886>
- [56] Minghao Xu, Jiarui Lu, Zuobai Zhang, Zhaocheng Zhu, Chang Ma, Yangtian Zhang, Runcheng Liu, and Jian Tang. 2022. PEER: A Comprehensive and Multi-Task Benchmark for Protein Sequence Understanding. In *Advances in Neural Information Processing Systems 35 (NeurIPS), Datasets and Benchmarks Track*.
- [57] Sumit Yadav. 2026. On the Relationship Between Representation Geometry and Generalization in Deep Neural Networks. *arXiv preprint arXiv:2602.00130* (2026). <https://arxiv.org/abs/2602.00130>
- [58] Shu-wen Yang, Po-Han Chi, Yung-Sung Chuang, Cheng-I Jeff Lai, Kushal Lakhotia, Yist Y. Lin, Andy T. Liu, Jiatong Shi, Xuankai Chang, Guan-Ting Lin, Tzu-Hsien Huang, Wei-Cheng Tseng, Ko-tik Lee, Da-Rong Liu, Zili Huang, Shuyan Dong, Shang-Wen Li, Shinji Watanabe, Abdelrahman Mohamed, and Hung-yi Lee. 2021. SUPERB: Speech Processing Universal PERFORMANCE Benchmark. In *Interspeech*.
- [59] Zhihan Yue, Yujing Wang, Juanyong Duan, Tianmeng Yang, Congrui Huang, Yunhai Tong, and Bixiong Xu. 2022. TS2Vec: Towards Universal Representation of Time Series. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 36. 8980–8987. <https://arxiv.org/abs/2106.10466>
- [60] Haoran Zhang, Yong Liu, Yunzhong Qiu, Haixuan Liu, Zhongyi Pei, Jianmin Wang, and Mingsheng Long. 2025. TimesBERT: A BERT-Style Foundation Model for Time Series Understanding. *arXiv preprint arXiv:2502.21245* (2025). <https://arxiv.org/abs/2502.21245>
- [61] Jinghao Zhou, Chen Wei, Huiyu Wang, Wei Shen, Cihang Xie, Alan Yuille, and Tao Kong. 2022. iBOT: Image BERT Pre-Training with Online Tokenizer. In *International Conference on Learning Representations (ICLR)*.
- [62] Kun Zhou, Hui Wang, Wayne Xin Zhao, Yutao Zhu, Sirui Wang, Fuzheng Zhang, Zhongyuan Wang, and Ji-Rong Wen. 2020. S³-Rec: Self-Supervised Learning for Sequential Recommendation with Mutual Information Maximization. In *Proceedings of the 29th ACM International Conference on Information and Knowledge Management*. <https://arxiv.org/abs/2008.07873>

Appendix

Contents

A	Related Work	11
B	Datasets	11
B.1	Dataset Sources	12
B.2	Preprocessing and Released Representation	12
C	Pretraining Method and Baseline Details	14
C.1	Masked Modelling	14
C.2	Joint-Embedding Predictive Learning	14
C.3	Contrastive Learning	14
C.4	Next-Token Prediction	15
C.5	Random Encoder Baselines	15
C.6	TSFresh Baseline	15
D	Method Hyperparameters	16
E	Full Results Tables	16
E.1	Per-Task Results	16
E.2	Aggregation Selection Details	16
E.3	Validator Sensitivity	18
E.4	Tucker-Based Feature Fusion	18
E.5	Embedding Fusion Analysis	24
E.6	Single- vs Multi-Objective Selection: Details	24
F	Data Splitting Protocol	25
G	Downstream Validator Details	27
H	Unsupervised Model-Selection Details	28

A Related Work

Reusing one representation across many downstream tasks is standard practice in many applications. Text and speech benchmarks evaluate a frozen encoder on a wide set of structurally different tasks [30, 58], PEER [56], which reports both frozen and fine-tuned settings, assembles seventeen protein tasks spanning function, localisation, structure, and interaction and BEACON [40] covers thirteen RNA tasks across structure, function, and engineering. Industrial systems rest on the same assumption, with a single learned user or item representation feeding ranking, retrieval, and downstream scoring [11, 19, 37]. In all of these settings, however, the encoder is optimised against a single signal, either the SSL loss or one downstream task, and only then evaluated on many.

The methods producing these representations draw on a small number of recurring concepts. Contrastive methods define invariance through augmentations or same-entity views [4, 23, 54, 59]; generative methods reconstruct hidden parts of the input, either from bidirectional context [13, 28, 35, 45, 51, 62] or under a causal mask, as in next-token prediction [2, 14, 25, 26] and latent-predictive methods regress representations rather than inputs [31]. Broadly, an SSL approach can be defined by three parts: (1) an augmentation strategy, (2) a loss function, and (3) a backbone. Each of these choices decides what the embedding must keep and what it may discard, however, views chosen for one task are not the best views for another [48], and the model class further biases which structure is recovered from the data [21]. For example, standard visual augmentations such as colour jitter force the encoder to ignore colour, and the resulting representations retain colour markedly less well than supervised ones [5, 15, 53]. One response is to combine concepts rather than choose between them, summing several objectives in a single pretraining stage [29, 61, 62]. We keep the

methods separate to make the per-task comparison interpretable and to establish the baseline any hybrid has to improve on.

Model Selection and Combining Representations. Some papers fix hyperparameters empirically or study released checkpoints [2, 18, 59]. Conclusions reached either way leave open whether the resulting ranking of methods is stable, or whether it would change under a different data split or if hyperparameters are optimized. Both EBES [34] and Time to Split [20] show that it can, a different split or hyperparameter optimization (HPO) is enough to reorder the methods. Neither, however, addresses selection when one embedding must serve several different tasks at once, a setting in which tuning per task also multiplies cost [36].

A more attractive option would be a single signal that reflects embedding quality across tasks, with such a criterion, an encoder could be optimised once rather than separately for every downstream task. Several measures have been proposed to assess representations directly: RankMe [17] uses effective rank, LiDAR [47] adapts rank estimation toward linear discriminability, aSMI [27] uses an information-theoretic criterion, and effective-dimensionality shows correlations with downstream accuracy [57]. Notably, this line of work is itself motivated by the observation that the SSL loss is uninformative about representation quality [17], so pretraining loss is already known to be a poor selection signal even in the single-task case. The proposed replacements, however, are validated against a single downstream endpoint, almost always linear-probe classification. Whether any of these criteria survives when one embedding must serve several tasks is untested, and it is what we evaluate in Section 5.4.

When no single encoder is adequate on several tasks at once, an alternative is to combine several. Classical subspace methods such as PCA [43] and CCA [7, 22] provide standard machinery for merging representations, and recent work revisits when such fusion helps [52]; tensor network models [10] like the Tucker decomposition [49] offer a further, more flexible alternative; we develop a Tucker-based fusion method in Section 5.6. This literature, however, has not been applied to the question of whether fused sequence embeddings can cover several downstream tasks simultaneously.

B Datasets

Table 7 summarises dataset structure and the complete downstream target matrix. Table 8 complements it with class balances, anomaly rates, and the horizon used for every target. Tables 9–11 report statistics of the released representation, exact model inputs, and the effective number of downstream examples.

We distinguish targets by their scope independently of the downstream task family. A global target assigns one value to an entire entity sequence and remains unchanged across its sampled slices. A local target is defined separately at each sampled cutoff using the subsequent dataset-specific horizon; different slices of the same sequence may therefore have different target values. All forecasting targets and all but one regression target are local, whereas classification and anomaly detection include both global and local targets. Table 8 reports the scope and horizon of every target.

Table 7: Dataset structure and downstream targets. [d] marks a default target and × means unavailable. Domains: TRX = transactions, RecSys = recommender systems, TS = time series, and Text = text. Row colors denote TRX (green), RecSys (blue), TS (orange), and Text (gray). Cardinality is the approximate number of categorical values.

Dataset	Domain	Structure	Features	Cardinality	Classification	Regression	Forecasting	Anomaly
AGE					Age group [d]			Payment-amount instability
AlphaBattle	TRX	Irregular	cat. + num.	50–1k categories	Application product, 5 classes	Purchase amount	Event count	Client default [d]
Retail					Quantile label [d]			Mostly loyalty-point payment
Taobao					Purchase/view quantile	Purchase amount		Purchase immediately after liking item
Zvuk					Future listening genre			Listening-duration variability
30Music	RecSys	Irregular	cat. + num.	100k–10M categories	Quantile of listening-time variability	Listening volume	Event count	High track diversity, low listening duration
Yambda					Recommended vs. organic listening [d]			High dislike-to-listen ratio
Electric Devices					Device type [d]	×	Energy change	×
ETT	TS	Regular	num.	Uni- and multivariate	×	×	Oil-temp change	×
Favorita					Store type, 5 classes	Sales volume	Item demand	Only store in city
Rossmann					Store type [d]		Sales demand	High sales max-to-median ratio
Twitter	Text	No timestamps	cat.	30–100 categories	Sentiment [d]	Mention count	×	Special-character ratio

Table 8: Classification and anomaly target details and target horizons. Class balances and anomaly rates are percentages. “G” denotes a global target, numeric values give local-target horizons, “step” denotes the next observation, and × means unavailable. Row colors denote TRX (green), RecSys (blue), TS (orange), and Text (gray).

Dataset	Domain	Classification target	Class balance	Anomaly target	Rate	Horizon			
						C	R	F	A
AGE		Age group, 4 classes [d]	24.8 : 25.0 : 25.2 : 25.0	Payment-amount coefficient of variation above the 95th percentile	5.00	G	30d	1d	G
AlphaBattle	TRX	Application product, 5 product IDs	51.3 : 29.8 : 6.2 : 7.9 : 4.7	Client default [d]	2.83	G	30d	1h	G
Retail		Age quantile, 4 classes [d]	26.8 : 22.1 : 26.9 : 24.1	Amount paid with loyalty points greater than 10	11.15	G	10d	1h	10d
Taobao		Purchase/view ratio, 2 quantiles	75 : 25	Item purchased immediately after being liked	4.59	48h	48h	1h	G
Zvuk		Future listening genre	Top 5: 27.97 : 6.23 : 5.31 : 3.54 : 2.83	Listening-duration coefficient of variation above the 95th percentile	5.00	40d	40d	1d	40d
30Music	RecSys	Listening-time variability, 4 classes	25 : 25 : 25 : 25	Track diversity > 0.95 and mean duration < 0.05	0.24	G	3d	1d	G
Yambda		Recommended vs. organic listening [d]	53.0 : 47.0	Dislike-to-listen ratio above the 95th percentile	5.01	G	10d	1h	G
Electric Devices		Device type [d]	26.98 : 25.02 : 16.50 : 9.54 : 8.14 : 8.13 : 5.69	×	×	G	×	1 step	×
ETT		×	×	×	×	×	×	1 step	×
Favorita	TS	Store type, 5 classes	33.96 : 28.30 : 15.09 : 15.09 : 7.55	Store is the only one in its city	21.00	G	30d	1d	G
Rossmann		Store type [d]	Top 5: 27.97 : 6.23 : 5.31 : 3.54 : 2.83	Max-to-median sales ratio over the next two months	5.00	G	60d	1d	60d
Twitter	Text	Sentiment [d]	54.03 : 31.13 : 13.30 : 1.54	Special-character count normalised by tweet length	2.00	G	G	×	G

B.1 Dataset Sources

The transaction datasets are AlphaBattle⁴, AGE⁵, and Retail⁶. The recommender-system datasets are Zvuk⁷, Yambda⁸, 30Music⁹, and Taobao¹⁰. The time-series datasets are Electric Devices¹¹, ETT¹², Favorita¹³, and Rossmann¹⁴. The text dataset is Twitter Sentiment¹⁵.

⁴<https://www.kaggle.com/datasets/mrmorj/alfabattle-20>

⁵<https://ods.ai/competitions/sberbank-sirius-lesson/data>

⁶<https://ods.ai/competitions/x5-retailhero-uplift-modeling/data>

⁷<https://www.kaggle.com/datasets/alex1/zvuk-dataset>

⁸<https://huggingface.co/datasets/yandex/yambda>

⁹<https://remaplab.deib.polimi.it/resources/>

¹⁰<https://tianchi.aliyun.com/dataset/46>

¹¹<https://www.timeseriesclassification.com/description.php?Dataset=ElectricDevices>

¹²<https://github.com/zhouhaoyi/ETDataset>

¹³<https://www.kaggle.com/competitions/favorita-grocery-sales-forecasting/data>

¹⁴<https://www.kaggle.com/competitions/rossmann-store-sales/data>

¹⁵<https://www.kaggle.com/datasets/kazanov/sentiment140/data>

B.2 Preprocessing and Released Representation

Unified on-disk representation. We release every dataset in the same entity-centric Parquet format. One row represents one entity (e.g., a customer, store, transformer, or tweet); its identifier and global targets are scalar columns, whereas time and event features are stored as aligned list-valued columns. Thus, the i th elements of the time, categorical, and numerical arrays describe the same event. The row also stores the sequence length, a list of valid cutoffs (shifts), and local-target arrays aligned one-to-one with those cutoffs. Target columns follow the self-describing convention: `target__<name>__<scope>__<metrics>` for example, `target__reg_amount__local__r2`. Train and test outputs are partitioned into Parquet shards, so the released files can be streamed without reconstructing sequences from the original event tables.

Table 9: Processed sequence and length statistics. “Sequences” is the number of rows and “Events” is the sum of `_seq_len`. Values are computed from the temporal test representation, whose rows contain full histories of entities with a valid post-boundary cutoff and therefore must not be added to temporal train counts. AlphaBattle combines its source-provided train and test partitions because they contain distinct entities. Twitter has only global targets and no temporal test representation, so its row reports the top-level training representation.

Dataset	Sequences	Events	Min	Mean	P90	P99	Max
AGE	29,964	26,420,179	700	881.73	1,071	1,143	1,150
AlphaBattle	1,457,032	443,407,241	6	304.32	734	1,539	9,580
Retail	212,322	33,490,385	6	157.73	299	560	2,513
Taobao	7,437	11,289,272	25	1,517.99	3,358.60	7,506.56	31,030
Zvuk	62,753	143,643,940	10	2,289.04	4,905.80	12,382	132,613
30Music	21,572	22,734,503	11	1,053.89	1,800	8,326.70	89,124
Yambda	8,280	46,355,169	11	5,598.45	13,913.80	24,482.21	27,699
Electric Devices	8,926	856,896	96	96.00	96	96	96
ETT	206	138,432	672	672.00	672	672	672
Favorita	54	90,936	1,684	1,684.00	1,684	1,684	1,684
Rossmann	1,115	1,017,209	758	912.30	942	942	942
Twitter	1,600,000	117,745,496	6	73.59	130	140	373

Table 10: Categorical and numerical inputs in the released representation. Categorical entries report the original cardinality; arrows denote cardinality caps applied by the model. “Not used” marks stored features excluded from model inputs, and “-” denotes an unavailable input type.

Dataset	Categorical features and cardinalities	Numerical model inputs
AGE	small_group: 202	amount_rur
AlphaBattle	card_type: 173; city: 163; country: 24; currency: 11; day_of_week: 7; ecommerce_flag: 3; income_flag: 3; mcc: 108; mcc_category: 28; operation_kind: 7; operation_type: 22; operation_type_group: 4; payment_system: 7; weekofyear: 53 (not used)	hour; amnt; hour_diff
Retail	is_alcohol: 2; is_own_trademark: 2; level_1: 3; level_2: 42; level_3: 201; level_4: 788; product_id: 42,522 (not used); segment_id: 116	trn_sum_from_iss; netto; regular_points_received; express_points_received; product_quantity; trn_sum_from_red; regular_points_spent; express_points_spent; purchase_sum
Taobao	behavior_type: 4; item_category: 8,916	-
Zvuk	cluster_id: 128; track_id: 1,506,950 → 20,000	play_duration
30Music	track_id: 4,519,104 → 15,000	play_duration
Yambda	event_type: 5; is_organic: 2; item_id: 934,057 → 15,000	played_ratio_pct; track_length_seconds
Electric Devices	-	sequence
ETT	-	HUFL; HULL; MUFL; MULL; LUFL; LULL
Favorita	class_id: 334 (not used)	class_1_sales-class_334_sales
Rossmann	DayOfWeek: 7; Open: 2; Promo: 2; SchoolHoliday: 2; StateHoliday: 1	Sales; Customers
Twitter	char: 190	-

Feature encoding. Categorical values are replaced with integer frequency ranks: code 1 is assigned to the most frequent value, larger codes to progressively rarer values, and 0 is reserved for missing or unknown categories. We release these mappings together with the processed data. All real-valued input features are normalised. Heavy-tailed quantities such as transaction amounts use a signed $\log(1+|x|)$ transform; other numerical channels are divided by a robust dataset-specific scale derived from the 0.99 quantile. Time is stored as a relative numerical coordinate rather than an absolute datetime. The reference point follows the source representation: AlphaBattle measures hours from each client’s first transaction; datasets supplied with calendar timestamps use offsets from a fixed dataset-level date; and regularly sampled series

use the observation index. The resulting time channel is rescaled together with the other numerical features. These transformations place heterogeneous numerical and temporal channels on comparable scales without discarding their within-sequence variation.

Shift construction and target storage. A shift s is a cutoff in an entity’s stored arrays: the encoder observes the prefix preceding the target window, and each local target is computed from observations after that cutoff. We retain only cutoffs with at least two preceding observations and enough future context to cover the task horizon in Table 8; entities without a valid cutoff are removed. From the remaining interval we sample up to a dataset-specific maximum number of shifts without replacement and store their sorted indices in the `shifts` column. Local targets are lists in the

Table 11: Downstream sample construction in the released representation. Shift counts are per-entity maxima; shorter histories may contribute fewer local examples. Global split counts are obtained from the `global_train` flag in the top-level training parquet and implement the approximately 90/10 entity split. For AlphaBattle, these global counts are computed within the source training partition, whereas its local test histories come from the source test partition. Dashes denote datasets without a target of the corresponding scope.

Dataset	Local tasks	Global tasks	Max shifts/entity		Global split	
			Train	Test	Train	Test
AGE	2	2	9	1	27,000	3,000
AlphaBattle	2	2	9	1	861,481	95,721
Retail	3	1	9	1	318,074	35,342
Taobao	3	1	9	1	8,749	973
Zvuk	4	0	9	1	–	–
30Music	2	2	9	1	35,934	3,993
Yambda	2	2	9	1	8,277	920
Electric Devices	1	1	27	3	8,033	893
ETT	1	0	90	10	–	–
Favorita	2	2	90	10	48	6
Rossmann	3	1	90	10	1,003	112
Twitter	0	3	0	–	1,440,000	160,000

same order, whereas a global target is stored once as a scalar because it applies to the complete entity sequence. This layout allows different history slices to be generated at loading time without duplicating the underlying event arrays. Table 11 reports the resulting task counts and shift limits.

C Pretraining Method and Baseline Details

This appendix details the implementations summarised in Table 2.

C.1 Masked Modelling

MLM trains a bidirectional encoder by hiding part of the input and reconstructing it from the remaining context [35, 45, 62]. In our event-sequence implementation, masking can select full events, spans, or individual categorical and numerical fields. Let M_c and M_r be the masked positions for categorical field c and numerical field r , let $\tilde{x}$ be the corrupted sequence, and let $\hat{x}_i^{(r)}$ be the reconstruction head prediction for numerical field r . The reconstruction terms are

$$\begin{aligned}\mathcal{L}_{\text{MLM-CE}} &= -\sum_c \frac{1}{|M_c|} \sum_{i \in M_c} \log p_\theta(x_i^{(c)} | \tilde{x}), \\ \mathcal{L}_{\text{MLM-MSE}} &= \sum_r \frac{1}{|M_r|} \sum_{i \in M_r} \left(\hat{x}_i^{(r)} - x_i^{(r)} \right)^2.\end{aligned}\quad (1)$$

The final MLM loss is a weighted sum of the categorical CE and numerical MSE terms. Masking creates explicit prediction targets: the model must recover the masked content itself. This differs from contrastive masking, where masked views are used as augmentations and the masked values are not reconstructed. Our search space varies whether complete events, contiguous spans, or individual categorical and numerical fields are masked, together with the probabilities of random replacement and leaving selected events unchanged. The downstream embedding is taken from the pre-trained bidirectional encoder after the reconstruction head is discarded.

C.2 Joint-Embedding Predictive Learning

JEPA predicts in representation space rather than reconstructing raw observations [3, 31]. Let C be the visible context positions, x_C the corresponding context events, and T the masked target positions. A context encoder E_θ maps the visible events to context tokens, a target encoder $E_{\hat{\theta}}$ maps the full sequence to target representations, and a predictor P_ϕ maps the context tokens, conditioned on target-position embeddings, into the target representation space. The target branch is stop-gradient and its parameters are not updated by backpropagation through the loss.

$$\begin{aligned}\mathcal{L}_{\text{JEPA}} &= \frac{1}{|T|} \sum_{i \in T} \text{SmoothL1}(p_i, \text{sg}(z_i)), \\ p_i &= P_\phi(E_\theta(x_C), i), \quad z_i = H_{\hat{\theta}}(x)_i.\end{aligned}\quad (2)$$

Here $\text{sg}(\cdot)$ denotes stop-gradient, $H_{\hat{\theta}}$ denotes the EMA target encoder followed by the target projection head, and p_i is the predictor output for target position i . The target parameters are updated by exponential moving average, $\hat{\theta} \leftarrow \mu \hat{\theta} + (1 - \mu)\theta$, after the online update. This is the defining JEPA mechanism for our benchmark: context encoder, EMA target encoder, predictor, and a latent-space target. We tune the EMA coefficient, predictor width, target projection, positional conditioning, and whether downstream embeddings are extracted from the online or target encoder.

C.3 Contrastive Learning

Our contrastive methods differ along two axes: how views of a sequence are generated and which contrastive loss is applied. CoLES uses same-entity random slices with the original margin loss. Slice-InfoNCE uses the same type of random slice views but replaces the margin loss with an InfoNCE objective. Mask-InfoNCE keeps the InfoNCE objective and replaces slicing with independently masked views.

C.3.1 CoLES [4]. builds positive pairs by sampling two subsequences from the same entity history and negative pairs from different entity histories. Let u and v be two sampled subsequences, $\Phi(\cdot)$ the sequence encoder, $c_u = \Phi(u)$ and $c_v = \Phi(v)$ their embeddings, $d_{uv} = \|c_u - c_v\|_2$, $Y_{uv} = 1$ for a positive same-entity pair and $Y_{uv} = 0$ otherwise, and ρ the margin for negative pairs. CoLES optimises the margin contrastive loss

$$\mathcal{L}_{uv}^{\text{CoLES}} = \frac{1}{2} Y_{uv} d_{uv}^2 + \frac{1}{2} (1 - Y_{uv}) \max(0, \rho - d_{uv})^2. \quad (3)$$

This objective pulls subsequences from the same entity together and pushes different-entity subsequences apart until the margin is satisfied. The loss therefore encourages stable entity-level information to be preserved across different temporal windows.

C.3.2 InfoNCE-based methods. Following the SimCLR two-view framework [8], the InfoNCE variants produce $2N$ views from a mini-batch of N sequences. Let h_i be the encoder output for view i , z_i the projection-head output used only for the contrastive loss, s_{ik} the cosine similarity between projected views i and k , (i, j) a positive pair of two views of the same sequence, and $\mathcal{A}(i) = \{1, \dots, 2N\} \setminus \{i\}$ the candidate set for anchor i :

$$s_{ik} = \frac{z_i^\top z_k}{\|z_i\|_2 \|z_k\|_2}. \quad (4)$$

The InfoNCE loss for anchor i and positive j is

$$\ell_{i,j}^{\text{InfoNCE}} = -\log \frac{\exp(s_{ij}/\tau)}{\exp(s_{ij}/\tau) + \sum_{k \in \mathcal{A}(i) \setminus \{j\}} \exp(s_{ik}/\tau)}. \quad (5)$$

The numerator is the positive pair; the denominator contains the positive pair and all other views in the batch as negatives. The final loss averages $\ell_{i,j}^{\text{InfoNCE}}$ over positive pairs. Our implementation also supports an angular margin on the positive similarity, but the objective family remains InfoNCE. Mask-InfoNCE is not masked reconstruction: it does not predict masked values, and masking only defines invariances between augmented views. After pretraining, the projection head is discarded and the encoder output is used as the downstream embedding.

C.4 Next-Token Prediction

NTP trains a causal model with a next-event prediction loss, as in next-item sequential recommendation models such as SASRec [26]. Let $x = (x_1, \dots, x_T)$ be an event sequence, let h_t be the hidden state produced from the prefix $x_{\leq t}$, and let $x_{t+1}^{(c)}$ and $x_{t+1}^{(r)}$ denote categorical and numerical fields of the next event. The next-event reconstruction terms are

$$\begin{aligned} \mathcal{L}_{\text{NTP-CE}} &= -\sum_{t=1}^{T-1} \sum_c \log p_\theta(x_{t+1}^{(c)} | x_{\leq t}), \\ \mathcal{L}_{\text{NTP-MSE}} &= \sum_{t=1}^{T-1} \sum_r (\hat{x}_{t+1}^{(r)} - x_{t+1}^{(r)})^2. \end{aligned} \quad (6)$$

The final loss is the weighted sum of CE and MSE terms over valid next-event positions. We implement NTP with two backbones: a causal Transformer/GPT-style encoder (NTP-GPT) or a GRU encoder (NTP-GRU). In both cases, the reconstruction head is used

only for pretraining and the hidden states are aggregated into downstream embeddings. NTP is used here only as a pretraining objective for embeddings, not as a downstream forecasting task.

C.5 Random Encoder Baselines

R-RNN and R-Tr are frozen, randomly initialised GRU and GPT-style Transformer encoders, respectively. They use the same input representation and embedding extraction rules as the corresponding pretrained encoders, isolating the contribution of SSL pretraining from that of the architecture and input encoding.

C.6 TSFresh Baseline

TSFresh [9] is included as a classical non-neural baseline. It maps each sequence to a fixed vector of hand-crafted time-series statistics rather than learning a parametric encoder. Our goal is not to tune a separate feature-engineering recipe per dataset, but to provide a strong, comparable classical reference under the same downstream protocol as the SSL methods.

Input representation. For every dataset we use the same fixed-order preprocessed splits as the neural methods. Each entity is identified by its native index column (`client_id`, `tweet_id`, `Store`, and so on). From the event-time axis we compute first-order inter-arrival gaps

$$\Delta t_i = t_i - t_{i-1}, \quad (7)$$

with $\Delta t_1 = 0$. For text-like sequences without timestamps, the same construction is applied to character-position indices. This yields one univariate series per entity (global mode) or one series per temporal cutoff (local mode).

Feature extraction. Features are computed with the library's EfficientFCParameters set. This configuration evaluates a broad catalogue of time-, frequency-, and signal-processing statistics while omitting the most expensive entropy-based calculators. For a single `time_delta` channel the resulting representation has fixed dimensionality 777; we keep *all* of these features.

TSFresh Feature-Importance Analysis. Although the benchmark uses the full 777-dimensional TSFresh vector, we additionally study *which* statistical families carry predictive signal. This analysis is separate from embedding construction: no features are removed before downstream evaluation.

For each dataset and downstream target, we fit an auxiliary gradient-boosted regressor (XGBRegressor) on the *training* TSFresh embeddings and record the top-25 features by importance. Targets are processed separately for global and local settings; for local targets, labels are aligned to the corresponding shift index.

We then aggregate these rankings across datasets and task families to identify recurrent feature types. In particular, we track how often FFT- and quantiles appear among the top predictors, and compare variants with and without explicit sequence-length statistics.

Figure 9 is constructed from one-hot encodings of the top TSFresh feature families per dataset-task pair. Hierarchical clustering of these profiles yields the dendrogram on the left; the stacked bars on the right show the corresponding family shares within each pair (normalized to sum to 1).

Five clusters emerge, and the same grouping is visible in both the dendrogram and the bar profiles. The first two clusters are dominated by regression-type targets (regression and forecasting). The next two are dominated by classification-type targets (classification and anomaly detection). The remaining cluster is more heterogeneous and sits farthest from the others in the hierarchy, suggesting a residual group whose top-feature composition differs most strongly from the rest.

D Method Hyperparameters

Training and stopping. Each HPO trial trains the encoder only on its SSL objective. CoLES, Slice-InfoNCE, Mask-InfoNCE, NTP-GRU, and NTP-GPT use early stopping on the SSL validation loss, with dataset-specific patience between three and ten epochs. MLM and JEPA instead run for a fixed 100 epochs. After the run terminates, the encoder is frozen and its embeddings are evaluated once on every available downstream task; these metrics form the multi-target Optuna objective. The downstream validators therefore select among completed runs but do not determine their training trajectories.

Compute budget. MLM and JEPA have larger conditional search spaces and use a time-bounded study: optimisation terminates at the study-level plateau criterion or after three days, resulting in approximately 150–300 trials depending on the dataset. Figure 10 provides a post-hoc convergence analysis for the fixed-budget NTP studies.

All neural methods tune learning rate η , weight decay λ , backbone width h , backbone depth L , categorical and numerical feature-embedding dimensions d_{cat} and d_{num} , and hidden-state pooling (mean or last). Table 12 gives their method-specific ranges together with the additional architectural, augmentation, and objective parameters. The subscript “log” denotes logarithmic sampling. The budget column reports the maximum number of trials or wall-clock limit, with the number of initial random trials in parentheses.

E Full Results Tables

E.1 Per-Task Results

Tables 14–17 report the per-dataset results underlying Table 1, one table per task family. Cells report the mean downstream metric and its standard deviation across repeated runs; the two random encoders (R-RNN, R-Tr) are shown in separated reference columns and do not participate in ranking. Superscripts show the positions occupied by each uncertainty-aware rank group over the eight non-random methods. Groups spanning two positions list both positions (e.g., 1, 2), whereas groups spanning three or more positions are written as a range (e.g., 1–4); each method receives the corresponding mid-rank when results are aggregated. Columns are ordered by the mean rank over the table’s datasets, reported in the Average rank row; per row, the best group is in bold and the second best is underlined. Row colors follow the domain grouping of Table 7. Degenerate settings are directly visible: on Favorita regression every pretrained encoder scores $R^2 < 0$ while the RNN random encoder does not, and on ETT forecasting only Slice-InfoNCE and Mask-InfoNCE improve over a constant predictor.

For completeness, Table 13 provides the complementary dataset-centred view. It ranks all ten methods within each dataset-task setting and then averages over the tasks available for each dataset.

Figure 11 visualises the same results as performance frontiers over the random baseline. For each dataset, the bar starts at the better of the two random encoders (grey); trained methods that beat this baseline are stacked in increasing order of score, so each colored segment is the incremental gain contributed by the method that reaches it, and the top of the stack is the best trained score. The label above each bar is the relative uplift of the best trained method over the best random encoder. Datasets where the stack barely rises above the grey base, most of forecasting and several regression datasets, are those where pretraining adds little over a random encoder, whereas on classification and anomaly detection the colored region is consistently tall. Several frontier segments are themselves thin: a method can rank first in a setting while improving only marginally over the next-best encoder, so a top rank does not necessarily imply a practically large gain.

Figure 12 complements the tables with the same uncertainty-aware ranks, aggregated overall and by task family. Each panel shows the mean rank of every method with one standard deviation across its datasets-task settings; methods are sorted within each panel. The overall panel matches the column averages of Table 13 up to weighting: here every dataset-task pair contributes equally, whereas Table 13 first averages within a dataset. The large standard deviations, often exceeding two rank positions, show that no method is consistently placed across datasets, and that single-dataset comparisons are unreliable for method selection.

E.2 Aggregation Selection Details

Figures 13 and 14 detail the aggregation choices summarised in Section 5.2.

At the method level (Figure 13), the methods included in this analysis follow the task-driven pattern, but with different strength. CoLES is a notable outlier: it is the only method that prefers the last hidden state even on globally targeted tasks (59% on global targets vs. 24–41% for the other methods), selecting it in 70% of anomaly and 64% of classification configurations. A plausible explanation is that CoLES is trained to make embeddings of random subsequences of the same user agree, so its recurrent state at any position, including the final one, is already optimised to be a sequence-level summary, and mean-pooling adds little. In contrast, NTP-GPT is the most mean-leaning method on global targets (18% last on classification, 20% on anomaly): its per-step states are optimised for next-event prediction, so entity-level information has to be recovered by averaging. Forecasting is unanimous: every analysed method selects the last state in at least 70% of configurations, and NTP-GRU in all of them.

At the dataset level (Figure 14), the choices track the semantics of individual targets. Twitter selects mean-pooling in every configuration across all tasks and methods included in this analysis, consistent with its lack of an event-time axis: all its targets are global sequence-level properties. Favorita anomaly detection is likewise unanimous for mean-pooling; its anomaly target (whether a store is the only one in its city) is a static property of the entity

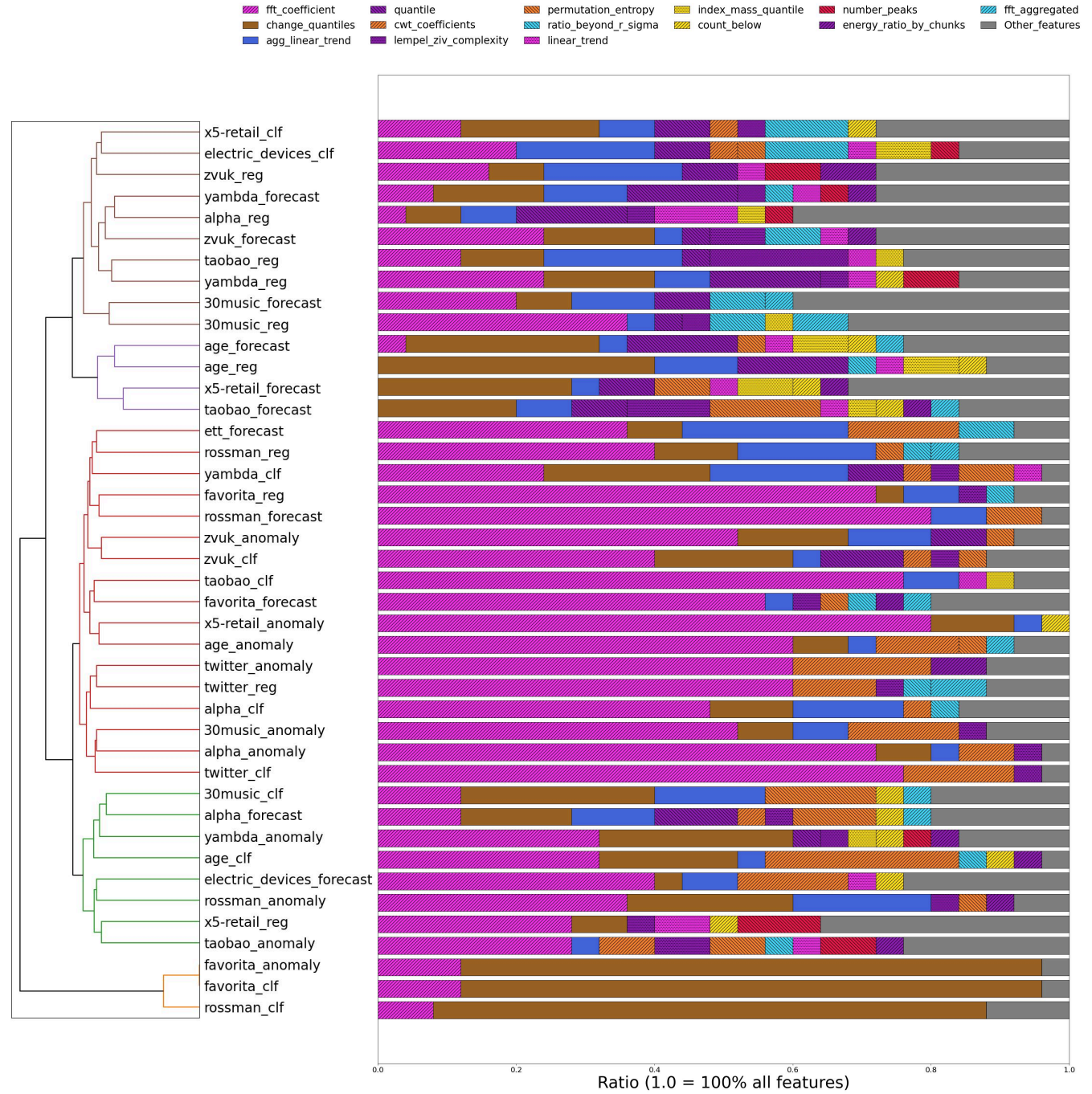


Figure 9: TSFresh feature-family analysis across dataset-task pairs. Left: hierarchical clustering of top-feature profiles. Right: normalized family shares within the top-25 XGB importances per pair (shares sum to 1 within each row). FFT- and quantile-related families recur across domains and tasks, but their relative weight varies by dataset and downstream objective.

rather than a temporal event. At the other extreme, AGE regression selects the last state for every analysed method, matching its short-horizon local target (purchase amount over the next days), and Taobao is the most recency-driven dataset overall (79% last

across tasks). The overall spread across datasets, from 0% to 79% last, shows that the aggregation choice reflects properties of the target definition rather than noise in hyperparameter search.

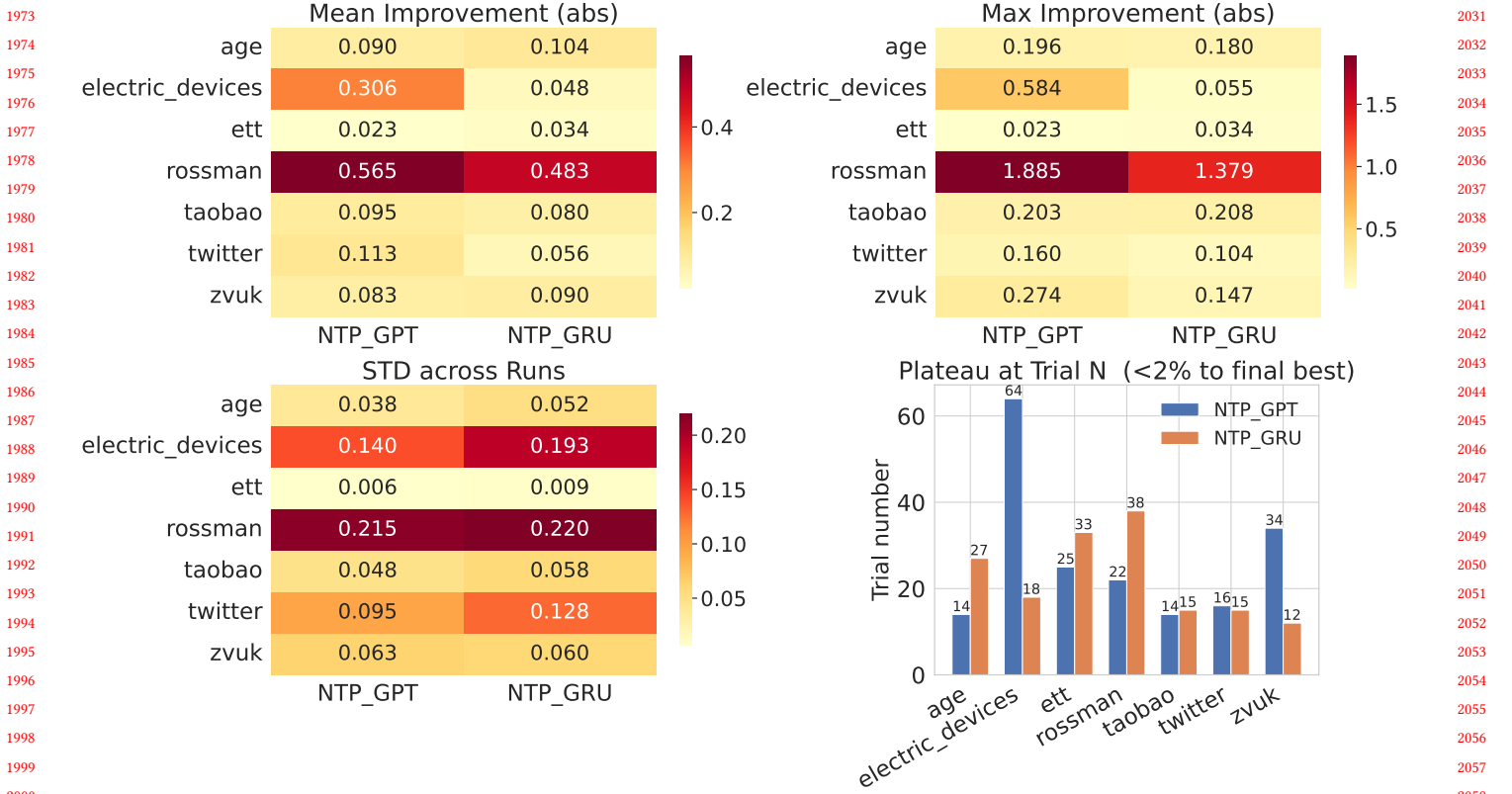


Figure 10: Post-hoc convergence analysis for the fixed-budget NTP studies. The heatmaps summarise the absolute downstream-metric improvement obtained during HPO and its variation across repeated studies; the lower-right panel reports the first trial whose best value is within 2% of the final study best. This diagnostic motivates the 70-trial NTP budget and is not itself an early-stopping rule.

E.3 Validator Sensitivity

The main text reports the final ten Optuna trials, which reflect configurations reached near the end of optimisation. Here the first ten, approximately random trials provide a control for the initial exploration stage. Table 18 compares pairwise validator correlations between these stages, and Table 19 reports the corresponding ranks and regret.

E.4 Tucker-Based Feature Fusion

Data Representation. Assume that each sample is represented by two feature vectors (views)

$$\mathbf{x}^{(n)} \in \mathbb{R}^{F_1}, \quad \mathbf{y}^{(n)} \in \mathbb{R}^{F_2}, \quad (8)$$

where $n = 1, \dots, N$. The training dataset is

$$\mathcal{D} = \left\{ \left(\mathbf{x}^{(n)}, \mathbf{y}^{(n)} \right) \right\}_{n=1}^N. \quad (9)$$

The objective is to learn compact representations that preserve the interactions between the two feature spaces.

Feature Interaction Model. We assume that every feature of the first view may interact with every feature of the second view. The

interaction is modeled by the outer product

$$\mathbf{X}^{(n)} = \mathbf{x}^{(n)} \otimes \mathbf{y}^{(n)}, \quad (10)$$

whose (i, j) -th entry is

$$X_{ij}^{(n)} = x_i^{(n)} y_j^{(n)}. \quad (11)$$

Thus each sample is represented by a rank-one fusion matrix

$$\mathbf{X}^{(n)} \in \mathbb{R}^{F_1 \times F_2}. \quad (12)$$

Collecting all samples forms the third-order fusion tensor

$$\mathcal{X} \in \mathbb{R}^{F_1 \times F_2 \times N}, \quad (13)$$

whose n -th frontal slice is

$$\mathcal{X}(:, :, n) = \mathbf{X}^{(n)} = \mathbf{x}^{(n)} \mathbf{y}^{(n)T}. \quad (14)$$

Problem Formulation. Since the fusion tensor can be extremely large, our objective is to find two projection matrices

$$\mathbf{B} \in \mathbb{R}^{F_1 \times r_1}, \quad \mathbf{C} \in \mathbb{R}^{F_2 \times r_2}, \quad (15)$$

where $r_1 \ll F_1$, $r_2 \ll F_2$, such that the interaction information contained in $\mathcal{X}$ is preserved as much as possible. The projections are obtained by computing a low-rank Tucker approximation

$$\mathcal{X} \approx \mathcal{G} \times_1 \mathbf{B} \times_2 \mathbf{C}, \quad (16)$$

Table 12: Hyperparameter search spaces. The middle column contains parameter groups shared by all neural methods; the next column contains only method-specific parameters and fixed design choices.

Method	Common search parameters	Additional parameters and fixed design	HPO budget
CoLES	$\eta : [10^{-5}, 10^{-2}]_{\log}; \lambda : [10^{-10}, 10^{-2}]_{\log}; h : [64, 1024]_{\log};$ $L : 1-5; d_{\text{cat}}, d_{\text{num}} : 1-128; \text{pool} \in \{\text{mean}, \text{last}\}$	Tuned: numerical normalisation on/off. Fixed: five slices, five hard negatives, margin $\rho = 1$.	100 (7 random)
Slice-InfoNCE	$\eta : [10^{-5}, 10^{-2}]_{\log}; \lambda : [10^{-10}, 10^{-2}]_{\log}; h : [64, 1024]_{\log};$ $L : 1-5; d_{\text{cat}}, d_{\text{num}} : 1-128; \text{pool} \in \{\text{mean}, \text{last}\}$	Tuned: temperature $\tau : [10^{-2}, 10^2]_{\log}$; time encoding $\in \{\text{event}, \Delta t\}$; numerical normalisation on/off. Fixed: two independently sampled slices.	100 (7 random)
Mask-InfoNCE	$\eta : [10^{-5}, 10^{-2}]_{\log}; \lambda : [10^{-10}, 10^{-2}]_{\log}; h : [64, 1024]_{\log};$ $L : 1-5; d_{\text{cat}}, d_{\text{num}} : 1-128; \text{pool} \in \{\text{mean}, \text{last}\}$	Tuned: temperature $\tau : [10^{-2}, 10^2]_{\log}$; time encoding $\in \{\text{event}, \Delta t\}$; numerical normalisation on/off. Fixed: two independently masked views.	100 (7 random)
MLM	$\eta : [10^{-5}, 10^{-2}]_{\log}; \lambda : [10^{-10}, 10^{-2}]_{\log}; h : [64, 1024]_{\log};$ $L : 1-5; d_{\text{cat}}, d_{\text{num}} : 16-128 \text{ (step 16)}; \text{pool} \in \{\text{mean}, \text{last}\}$	Architecture: heads $\in \{1, 2, 4, 8\}$; block passes $\in \{1, 2\}$; dropout $[0, 0.3]$; output size $\in \{64, 256, 1024\}$. Masking: probability $[0.1, 0.8]$; span length 5–100; span share $[0.1, 0.9]$; random replacement $[0, 0.2]$; unchanged event $[0, 0.15]$; partial categorical/numerical masking $[0, 0.5]$.	≤ 3 days (10 random)
JEP A	$\eta : [10^{-5}, 10^{-2}]_{\log}; \lambda : [10^{-10}, 10^{-2}]_{\log}; h : [128, 1024]_{\log};$ $L : 1-4; d_{\text{cat}}, d_{\text{num}} : 32-128 \text{ (step 16)}; \text{pool} \in \{\text{mean}, \text{last}\}$	Architecture: heads $\in \{1, 2, 4, 8\}$; block passes $\in \{1, 2\}$; dropout $[0, 0.2]$; output size $\in \{64, 256, 1024\}$. JEP A: EMA $\in \{0.99, 0.999, 0.9999\}$; downstream branch $\in \{\text{online}, \text{target}\}$; predictor multiplier $\in \{0.5, 1, 2\}$; target projection $\in \{\text{predictor output}, \text{online target}\}$; mask probability $[0.1, 0.8]$.	≤ 3 days (10 random)
NTP-GRU	$\eta : [10^{-5}, 10^{-2}]_{\log}; \lambda : [10^{-10}, 10^{-2}]_{\log}; h : [64, 1024]_{\log};$ $L : 1-5; d_{\text{cat}}, d_{\text{num}} : 8-128 \text{ (step 8)}; \text{pool} \in \{\text{mean}, \text{last}\}$	Numerical normalisation on/off; output size $\in \{64, 256, 1024\}$; GRU backbone.	70 (7 random)
NTP-GPT	$\eta : [10^{-5}, 10^{-2}]_{\log}; \lambda : [10^{-10}, 10^{-2}]_{\log}; h : [64, 1024]_{\log};$ $L : 1-5; d_{\text{cat}}, d_{\text{num}} : 8-128 \text{ (step 8)}; \text{pool} \in \{\text{mean}, \text{last}\}$	Numerical normalisation on/off; output size $\in \{64, 256, 1024\}$; heads $\in \{1, 2, 4, 8\}$; GPT-style causal Transformer backbone.	70 (7 random)

Table 13: Average uncertainty-aware rank of each method per dataset (lower is better), averaged over the downstream tasks available for that dataset. Methods are ordered by their column average; the best value per row is in bold, the second best is underlined. Row colors group datasets by domain as in Table 7: transactions (green), recommender systems (blue), time series (orange), text (gray).

Dataset	CoLES	Slice-InfoNCE	NTP-GRU	NTP-GPT	Mask-InfoNCE	JEP A	MLM	TSFresh	Random-Tr	Random
AGE	2.50	<u>2.75</u>	2.88	3.88	5.50	5.25	6.38	9.50	8.62	7.75
AlphaBattle	2.50	5.50	<u>2.62</u>	4.12	8.62	4.50	6.50	5.62	7.38	7.62
Retail	1.62	<u>2.38</u>	3.88	7.38	3.00	5.12	7.00	8.12	7.75	8.75
Taobao	1.75	<u>2.12</u>	4.75	6.75	7.25	3.62	5.12	8.50	6.25	8.88
Zvuk	1.75	<u>1.88</u>	6.38	4.38	5.62	8.38	5.88	6.00	5.12	9.62
30Music	2.12	6.75	5.75	5.88	4.88	<u>2.88</u>	6.38	4.50	7.38	8.50
Yambda	4.25	2.88	4.38	4.50	<u>3.50</u>	5.62	6.88	5.38	7.75	9.88
Rossmann	2.00	4.25	5.88	4.88	<u>3.25</u>	3.88	4.38	9.00	8.88	8.62
ETT	6.50	1.00	6.50	3.50	<u>2.00</u>	3.50	6.50	9.00	10.00	6.50
Electric Devices	3.00	3.50	<u>2.75</u>	4.00	2.50	8.25	6.50	7.25	9.00	8.25
Favorita	5.00	6.88	3.50	<u>3.62</u>	6.88	3.50	6.62	6.38	6.00	6.62
Twitter	4.00	2.33	<u>2.67</u>	2.83	3.17	7.00	6.33	9.00	9.33	8.33
Average	3.08	<u>3.52</u>	4.33	4.64	4.68	5.12	6.20	7.35	7.79	8.28

where $\mathcal{G} \in \mathbb{R}^{r_1 \times r_2 \times N}$ is the Tucker core tensor.

The corresponding optimization problem is

$$\begin{aligned}
 \min_{\mathbf{B}, \mathbf{C}, \mathcal{G}} \quad & \|\mathbf{X} - \mathcal{G} \times_1 \mathbf{B} \times_2 \mathbf{C}\|_F^2, \\
 \text{s.t.} \quad & \mathbf{B}^T \mathbf{B} = \mathbf{I}, \\
 & \mathbf{C}^T \mathbf{C} = \mathbf{I}.
 \end{aligned} \tag{17}$$

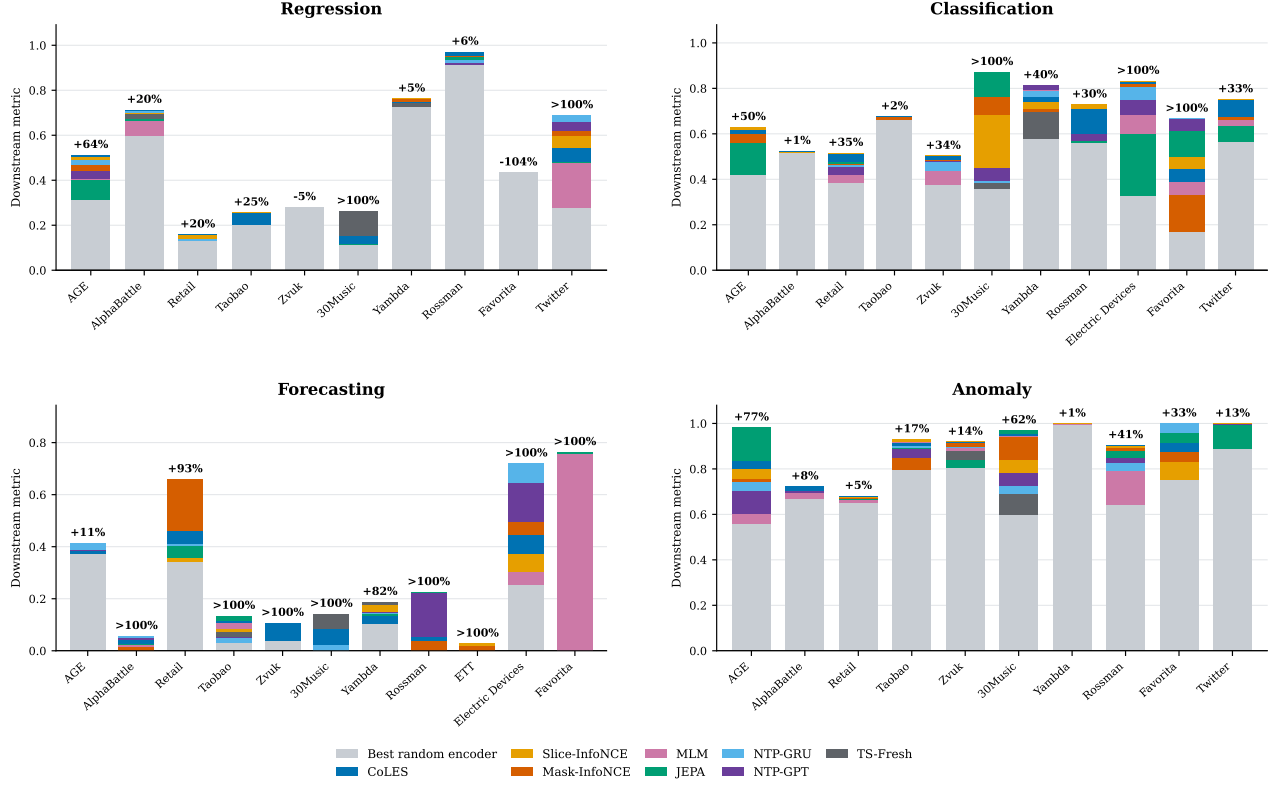


Figure 11: Performance frontiers above the random-encoder baseline, one panel per task family (companion to Tables 14–17). Each bar starts at the better of the two random encoders (grey); trained methods beating it are stacked in increasing order, so each colored segment is the incremental gain to the score reached by that method and the top of the stack is the best trained score. Labels show the relative uplift of the best trained method over the best random encoder.

Table 14: Regression: absolute R^2 of each method per dataset (higher is better). Conventions are described in Appendix E.1.

Regression	R-RNN	R-Tr	CoLES	Slice-InfoNCE	NTP-GRU	Mask-InfoNCE	TSFresh	NTP-GPT	JEPA	MLM
AGE	0.306 ± 0.001	0.313 ± 0.002	0.512 ± 0.001 ¹	0.506 ± 0.001 ²	0.490 ± 0.001 ³	0.471 ± 0.002 ⁴	0.223 ⁸	0.440 ± 0.001 ⁵	0.401 ± 0.004 ^{6,7}	0.405 ± 0.002 ^{6,7}
AlphaBattle	0.393 ± 0.001	0.596 ± 0.005	0.713 ± 0.002 ¹	0.699 ± 0.000 ³	0.710 ± 0.000 ²	0.666 ± 0.001 ⁶⁻⁸	0.697 ⁴	0.667 ± 0.001 ⁶⁻⁸	0.672 ± 0.002 ⁵	0.665 ± 0.002 ⁶⁻⁸
Retail	0.041 ± 0.000	0.132 ± 0.001	0.159 ± 0.004 ^{1,2}	0.159 ± 0.001 ^{1,2}	0.138 ± 0.002 ^{3,4}	0.133 ± 0.004 ^{3,4}	0.129 ⁵	0.105 ± 0.004 ⁶	0.062 ± 0.009 ⁸	0.089 ± 0.005 ⁷
Taobao	-0.021 ± 0.005	0.205 ± 0.005	0.254 ± 0.008 ^{1,2}	0.257 ± 0.002 ^{1,2}	0.191 ± 0.002 ^{3,4}	0.007 ± 0.003 ⁷	-0.201 ⁸	0.074 ± 0.003 ⁶	0.179 ± 0.012 ^{3,4}	0.103 ± 0.023 ⁵
Zvuk	-0.016 ± 0.000	0.281 ± 0.001	0.252 ± 0.010 ^{1,2}	0.266 ± 0.005 ^{1,2}	0.131 ± 0.001 ⁶	-0.028 ± 0.014 ⁸	0.189 ³	0.172 ± 0.003 ^{4,5}	0.036 ± 0.002 ⁷	0.162 ± 0.007 ^{4,5}
30Music	0.002 ± 0.001	0.113 ± 0.003	0.152 ± 0.001 ²	0.006 ± 0.002 ⁷	0.051 ± 0.002 ⁵	0.066 ± 0.003 ⁴	0.263 ¹	0.034 ± 0.003 ⁶	0.117 ± 0.011 ³	-0.020 ± 0.022 ⁸
Yambda	0.580 ± 0.001	0.727 ± 0.002	0.749 ± 0.001 ³	0.763 ± 0.001 ¹	0.686 ± 0.005 ⁶	0.762 ± 0.000 ²	0.745 ⁴	0.699 ± 0.005 ⁵	0.674 ± 0.001 ⁷	0.611 ± 0.003 ⁸
Rossmann	0.915 ± 0.001	-0.084 ± 0.049	0.971 ± 0.001 ¹	0.951 ± 0.000 ⁴	0.936 ± 0.001 ⁶	0.953 ± 0.000 ³	-0.031 ⁸	0.922 ± 0.002 ⁷	0.947 ± 0.000 ⁵	0.954 ± 0.000 ²
Favorita	0.163 ± 0.043	0.435 ± 0.054	-0.742 ± 0.129 ^{3,4}	-1.421 ± 0.000 ⁵⁻⁷	-0.154 ± 0.017 ²	-1.304 ± 0.128 ⁵⁻⁷	-0.016 ¹	-0.781 ± 0.054 ^{3,4}	-1.447 ± 0.503 ⁵⁻⁷	-2.393 ± 0.805 ⁸
Twitter	0.280 ± 0.002	0.067 ± 0.000	0.545 ± 0.000 ⁵	0.598 ± 0.000 ⁴	0.689 ± 0.000 ¹	0.619 ± 0.000 ³	-0.017 ⁸	0.659 ± 0.000 ²	0.480 ± 0.000 ⁶	0.478 ± 0.000 ⁷
Avg. rank	-	-	2.1	3.1	3.8	4.8	5.0	5.2	5.7	6.3

Equivalent Optimization. For fixed projection matrices, the optimal Tucker core is

$$\mathcal{G} = \mathcal{X} \times_1 \mathbf{B}^T \times_2 \mathbf{C}^T. \quad (18)$$

Substituting the optimal core into (17) yields the equivalent maximization problem

$$\begin{aligned} \max_{\mathbf{B}, \mathbf{C}} \quad & \|\mathcal{X} \times_1 \mathbf{B}^T \times_2 \mathbf{C}^T\|_F^2, \\ \text{s.t.} \quad & \mathbf{B}^T \mathbf{B} = \mathbf{I}, \\ & \mathbf{C}^T \mathbf{C} = \mathbf{I}. \end{aligned} \quad (19)$$

Table 15: Classification: absolute accuracy of each method per dataset (higher is better). Conventions are described in Appendix E.1.

Classification	R-RNN	R-Tr	Slice-InfoNCE	CoLES	NTP-GPT	NTP-GRU	Mask-InfoNCE	JEPA	MLM	TSFresh
AGE	0.399 ±0.003	0.420 ±0.010	0.630 ±0.004 ¹	0.617 ±0.002 ⁴	0.620 ±0.003 ^{2,3}	0.621 ±0.001 ^{2,3}	0.601 ±0.003 ^{5,6}	0.558 ±0.002 ⁷	0.601 ±0.002 ^{5,6}	0.418 ⁸
AlphaBattle	0.517 ±0.002	0.513 ±0.005	0.523 ±0.009 ¹⁻⁵	0.524 ±0.010 ¹⁻⁵	0.518 ±0.003 ¹⁻⁵	0.517 ±0.004 ¹⁻⁵	0.508 ±0.000 ⁸	0.517 ±0.004 ¹⁻⁵	0.510 ±0.005 ^{6,7}	0.512 ^{6,7}
Retail	0.372 ±0.006	0.384 ±0.007	0.516 ±0.004 ^{1,2}	0.513 ±0.004 ^{1,2}	0.458 ±0.001 ⁶	0.463 ±0.001 ⁵	0.469 ±0.002 ⁴	0.479 ±0.005 ³	0.421 ±0.005 ⁷	0.315 ⁸
Taobao	0.663 ±0.000	0.663 ±0.000	0.676 ±0.000 ²	0.676 ±0.000 ¹	0.654 ±0.017 ⁶	0.676 ±0.000 ³⁻⁵	0.676 ±0.000 ³⁻⁵	0.676 ±0.000 ³⁻⁵	0.562 ±0.034 ⁷	0.378 ⁸
Zvuk	0.278 ±0.000	0.377 ±0.002	0.503 ±0.001 ^{1,2}	0.502 ±0.001 ^{1,2}	0.479 ±0.002 ⁴	0.476 ±0.000 ⁵	0.486 ±0.003 ³	0.339 ±0.000 ⁸	0.438 ±0.002 ⁶	0.364 ⁷
30Music	0.290 ±0.006	0.359 ±0.005	0.683 ±0.006 ⁴	0.765 ±0.005 ^{2,3}	0.449 ±0.001 ⁶	0.392 ±0.004 ⁷	0.761 ±0.002 ^{2,3}	0.872 ±0.004 ¹	0.454 ±0.003 ⁵	0.386 ⁸
Yambda	0.572 ±0.015	0.581 ±0.014	0.742 ±0.006 ⁵	0.761 ±0.004 ⁴	0.814 ±0.006 ¹	0.790 ±0.015 ^{2,3}	0.711 ±0.007 ^{6,7}	0.711 ±0.007 ^{6,7}	0.795 ±0.004 ^{2,3}	0.696 ⁸
Rossmann	0.560 ±0.005	0.562 ±0.009	0.729 ±0.022 ¹⁻³	0.708 ±0.005 ¹⁻³	0.601 ±0.027 ⁴⁻⁷	0.568 ±0.010 ⁴⁻⁷	0.708 ±0.019 ¹⁻³	0.568 ±0.010 ⁴⁻⁷	0.568 ±0.010 ⁴⁻⁷	0.527 ⁸
Electric Devices	0.329 ±0.029	0.323 ±0.016	0.833 ±0.002 ¹⁻³	0.832 ±0.005 ¹⁻³	0.751 ±0.006 ⁶	0.807 ±0.011 ^{4,5}	0.820 ±0.013 ¹⁻³	0.603 ±0.014 ⁸	0.683 ±0.001 ⁷	0.807 ^{4,5}
Favorita	0.167 ±0.167	0.167 ±0.289	0.500 ±0.000 ^{4,5}	0.444 ±0.255 ^{4,5}	0.667 ±0.289 ¹⁻³	0.667 ±0.167 ¹⁻³	0.333 ±0.000 ^{6,7}	0.611 ±0.096 ¹⁻³	0.389 ±0.096 ^{6,7}	0.000 ⁸
Twitter	0.564 ±0.004	0.551 ±0.010	0.752 ±0.000 ^{1,2}	0.747 ±0.000 ⁴	0.752 ±0.001 ^{1,2}	0.751 ±0.000 ³	0.672 ±0.001 ⁵	0.634 ±0.000 ⁸	0.662 ±0.001 ⁶	0.636 ⁷
Avg. rank	–	–	2.5	2.7	4.0	4.0	4.5	5.1	5.9	7.4

Table 16: Forecasting: absolute R^2 of each method per dataset (higher is better). Conventions are described in Appendix E.1.

Forecasting	R-RNN	R-Tr	NTP-GPT	CoLES	JEPA	NTP-GRU	Slice-InfoNCE	TSFresh	Mask-InfoNCE	MLM
AGE	0.373 ±0.001	0.292 ±0.001	0.390 ±0.004 ²	0.385 ±0.001 ³	0.330 ±0.007 ^{5,6}	0.414 ±0.001 ¹	0.344 ±0.001 ⁴	-0.249 ⁸	0.292 ±0.003 ⁷	0.327 ±0.001 ^{5,6}
AlphaBattle	-0.006 ±0.003	-0.004 ±0.003	0.050 ±0.000 ²	0.042 ±0.000 ⁴	0.025 ±0.006 ^{5,6}	0.057 ±0.001 ¹	0.028 ±0.003 ^{5,6}	0.044 ³	0.017 ±0.001 ⁸	0.023 ±0.003 ⁷
Retail	0.342 ±0.003	0.260 ±0.008	0.298 ±0.002 ^{6,7}	0.460 ±0.000 ²	0.403 ±0.108 ^{3,4}	0.412 ±0.002 ^{3,4}	0.359 ±0.002 ⁵	0.081 ⁸	0.659 ±0.004 ¹	0.310 ±0.029 ^{6,7}
Taobao	-0.004 ±0.002	0.033 ±0.003	0.052 ±0.008 ^{6,7}	0.116 ±0.000 ²	0.134 ±0.007 ¹	0.049 ±0.000 ^{6,7}	0.087 ±0.002 ⁴	0.074 ⁵	-0.021 ±0.002 ⁸	0.107 ±0.006 ³
Zvuk	-0.135 ±0.011	0.040 ±0.002	0.016 ±0.001 ^{2,3}	0.107 ±0.002 ¹	-0.098 ±0.127 ^{7,8}	-0.097 ±0.001 ^{7,8}	0.031 ±0.017 ^{2,3}	0.012 ⁴	-0.060 ±0.019 ⁶	-0.001 ±0.003 ⁵
30Music	-0.005 ±0.000	-0.032 ±0.006	-0.003 ±0.004 ⁴	0.085 ±0.002 ²	-0.019 ±0.002 ⁵	0.024 ±0.004 ³	-0.054 ±0.002 ⁸	0.142 ¹	-0.041 ±0.012 ^{6,7}	-0.033 ±0.007 ^{6,7}
Yambda	0.081 ±0.001	0.103 ±0.003	0.149 ±0.002 ⁴⁻⁶	0.135 ±0.004 ⁷	0.144 ±0.005 ⁴⁻⁶	0.148 ±0.004 ⁴⁻⁶	0.177 ±0.002 ^{2,3}	0.188 ¹	0.177 ±0.004 ^{2,3}	0.085 ±0.002 ⁸
Rossmann	-1.114 ±0.000	-0.500 ±0.000	0.222 ±0.000 ²	0.053 ±0.000 ⁴	0.225 ±0.000 ¹	-0.018 ±0.000 ⁶	-0.606 ±0.000 ⁸	-0.057 ⁷	0.040 ±0.000 ⁵	0.055 ±0.000 ³
ETT	-0.011 ±0.005	-0.399 ±0.212	-0.007 ±0.002 ^{3,4}	-0.012 ±0.007 ³⁻⁷	-0.009 ±0.002 ^{3,4}	-0.010 ±0.001 ⁵⁻⁷	0.028 ±0.000 ¹	-0.045 ⁸	0.017 ±0.004 ²	-0.010 ±0.002 ⁵⁻⁷
Electric Devices	0.253 ±0.001	0.170 ±0.047	0.645 ±0.001 ²	0.447 ±0.009 ⁴	0.216 ±0.006 ⁷	0.720 ±0.001 ¹	0.372 ±0.004 ⁵	-0.008 ⁸	0.497 ±0.002 ³	0.304 ±0.008 ⁶
Favorita	-1.821 ±0.000	-1.713 ±0.001	-0.884 ±0.001 ⁴	-1.745 ±0.000 ⁶	0.764 ±0.000 ¹	-1.333 ±0.000 ⁵	-1.770 ±0.000 ⁷	-0.039 ³	-1.786 ±0.000 ⁸	0.759 ±0.001 ²
Avg. rank	–	–	3.6	3.7	4.1	4.1	4.8	5.1	5.2	5.3

Table 17: Anomaly detection: absolute ROC-AUC of each method per dataset (higher is better). Conventions are described in Appendix E.1.

Anomaly	R-RNN	R-Tr	CoLES	Slice-InfoNCE	Mask-InfoNCE	JEPA	NTP-GRU	NTP-GPT	MLM	TSFresh
AGE	0.556 ±0.005	0.513 ±0.024	0.836 ±0.002 ²	0.800 ±0.004 ³	0.757 ±0.003 ⁴	0.982 ±0.000 ¹	0.745 ±0.002 ⁵	0.701 ±0.003 ⁶	0.604 ±0.010 ⁷	0.501 ⁸
AlphaBattle	0.645 ±0.025	0.668 ±0.013	0.723 ±0.003 ¹	0.627 ±0.005 ^{7,8}	0.613 ±0.039 ^{7,8}	0.696 ±0.014 ²⁻⁵	0.705 ±0.005 ²⁻⁵	0.701 ±0.019 ²⁻⁵	0.694 ±0.008 ²⁻⁵	0.658 ⁶
Retail	0.619 ±0.001	0.651 ±0.002	0.681 ±0.003 ^{1,2}	0.680 ±0.006 ^{1,2}	0.673 ±0.002 ^{3,4}	0.667 ±0.002 ⁵	0.674 ±0.002 ^{3,4}	0.649 ±0.001 ⁸	0.664 ±0.002 ^{6,7}	0.665 ^{6,7}
Taobao	0.700 ±0.003	0.797 ±0.002	0.917 ±0.001 ^{2,3}	0.930 ±0.001 ¹	0.849 ±0.003 ⁷	0.894 ±0.002 ⁵	0.900 ±0.001 ⁴	0.889 ±0.001 ⁶	0.919 ±0.002 ^{2,3}	0.776 ⁸
Zvuk	0.803 ±0.002	0.794 ±0.000	0.918 ±0.000 ²	0.920 ±0.000 ¹	0.914 ±0.000 ³	0.841 ±0.003 ⁸	0.895 ±0.000 ⁵	0.899 ±0.001 ⁴	0.890 ±0.003 ⁶	0.877 ⁷
30Music	0.479 ±0.107	0.600 ±0.011	0.955 ±0.001 ²	0.840 ±0.001 ⁵	0.941 ±0.002 ⁴	0.970 ±0.002 ¹	0.726 ±0.005 ⁷	0.782 ±0.004 ⁶	0.946 ±0.001 ³	0.691 ⁸
Yambda	0.987 ±0.003	0.993 ±0.002	0.999 ±0.000 ¹⁻⁵	0.999 ±0.000 ¹⁻⁵	0.999 ±0.000 ¹⁻⁵	0.999 ±0.001 ¹⁻⁵	0.999 ±0.000 ¹⁻⁵	0.999 ±0.000 ⁶	0.997 ±0.000 ⁷	0.991 ⁸
Rossmann	0.643 ±0.002	0.562 ±0.024	0.904 ±0.001 ¹	0.901 ±0.001 ²	0.893 ±0.001 ³	0.882 ±0.001 ⁴	0.827 ±0.001 ⁶	0.849 ±0.000 ⁵	0.792 ±0.000 ⁷	0.452 ⁸
Favorita	0.750 ±0.331	0.500 ±0.217	0.917 ±0.144 ¹⁻⁵	0.833 ±0.006 ^{6,7}	0.875 ±0.217 ¹⁻⁵	0.958 ±0.072 ¹⁻⁵	1.000 ±0.000 ¹⁻⁵	0.917 ±0.144 ¹⁻⁵	0.667 ±0.577 ^{6,7}	0.500 ⁸
Twitter	0.888 ±0.001	0.714 ±0.001	0.998 ±0.000 ³	0.999 ±0.000 ^{1,2}	0.999 ±0.000 ^{1,2}	0.994 ±0.000 ⁷	0.998 ±0.000 ⁴	0.997 ±0.000 ⁵	0.996 ±0.000 ⁶	0.638 ⁸
Avg. rank	–	–	2.1	3.2	4.0	4.0	4.4	5.2	5.5	7.5

Table 18: Pairwise validator correlations by dataset for the first and final ten Optuna trials (LP is a linear probe).

Trials	Pair	AGE	Taobao	Twitter	Retail	Yambda	Zvuk
First 10	LightGBM-MLP	0.981	0.779	0.459	0.905	0.920	0.298
First 10	LP-LightGBM	0.976	0.765	0.226	0.864	0.870	0.451
First 10	LP-MLP	0.989	0.764	0.421	0.944	0.859	0.611
Final 10	LightGBM-MLP	0.900	0.667	0.494	0.970	0.758	0.581
Final 10	LP-LightGBM	0.917	0.513	0.495	0.944	0.764	0.554
Final 10	LP-MLP	0.928	0.680	0.918	0.970	0.684	0.595

This formulation seeks the pair of orthonormal projection matrices that maximizes the energy preserved in the projected fusion tensor.

Efficient Solution. Rather than explicitly constructing the tensor $\mathcal{X}$, the Tucker decomposition can be computed from its mode-wise Gram matrices.

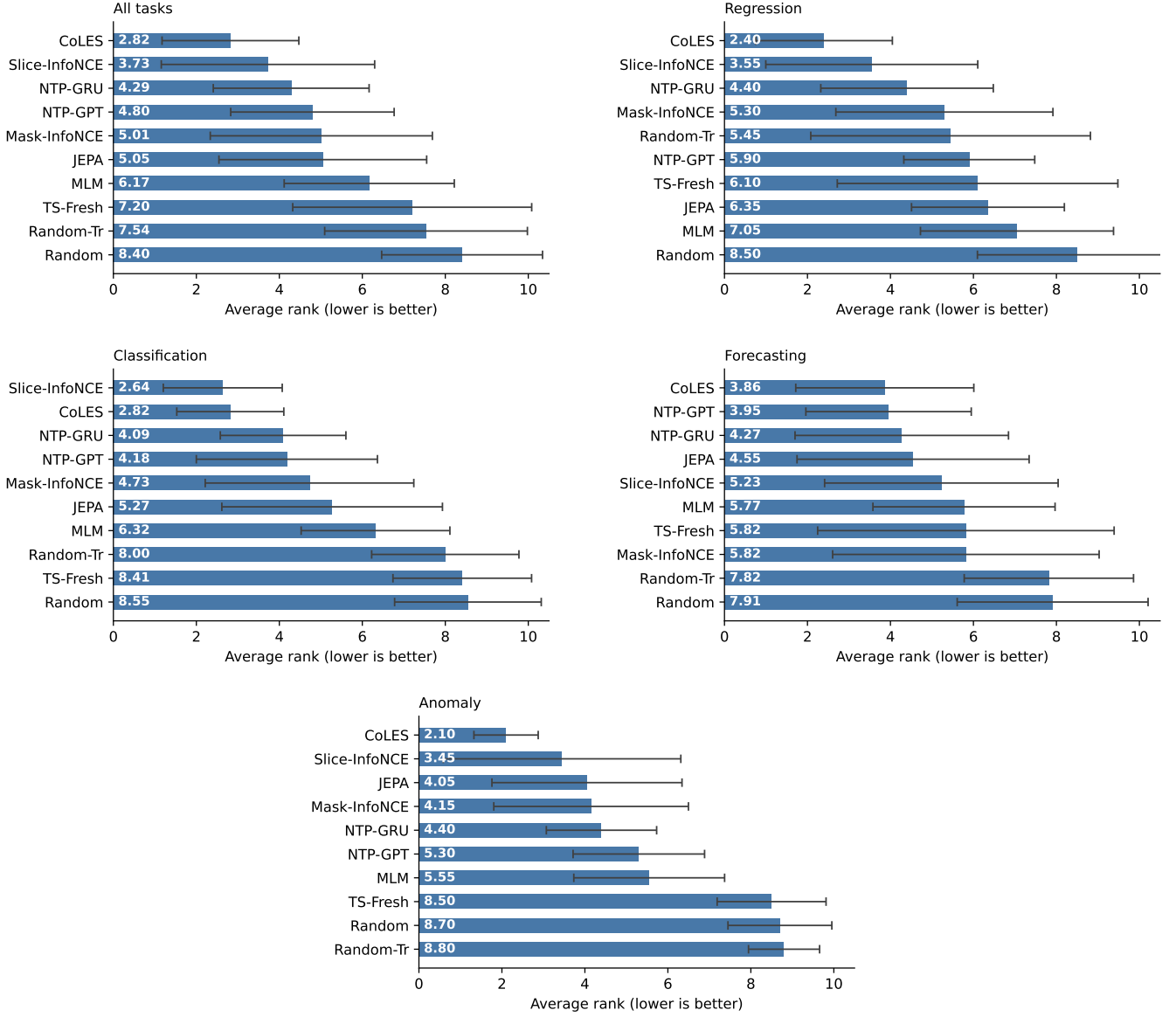


Figure 12: Uncertainty-aware ranks of all ten methods (lower is better), shown as the mean and one standard deviation across all 42 dataset-task settings and separately for each task family. Methods are sorted by mean rank within each panel; ranks use the same mean-performance intervals and tie groups as Table 13.

The mode-1 Gram matrix is

$$\begin{aligned}
 \mathbf{G}_1 &= \mathbf{X}_{(1)} \mathbf{X}_{(1)}^T \\
 &= \sum_{n=1}^N \mathbf{x}^{(n)} \mathbf{y}^{(n)T} \mathbf{y}^{(n)} \mathbf{x}^{(n)T} \\
 &= \sum_{n=1}^N \|\mathbf{y}^{(n)}\|^2 \mathbf{x}^{(n)} \mathbf{x}^{(n)T}.
 \end{aligned} \tag{20}$$

Similarly,

$$\mathbf{G}_2 = \sum_{n=1}^N \|\mathbf{x}^{(n)}\|^2 \mathbf{y}^{(n)} \mathbf{y}^{(n)T}. \tag{21}$$

The projection matrices are obtained from the dominant eigenvectors

$$\mathbf{B} = \text{eig}_{r_1}(\mathbf{G}_1), \quad \mathbf{C} = \text{eig}_{r_2}(\mathbf{G}_2), \tag{22}$$

where $\text{eig}_r(\cdot)$ denotes the eigenvectors corresponding to the r largest eigenvalues.

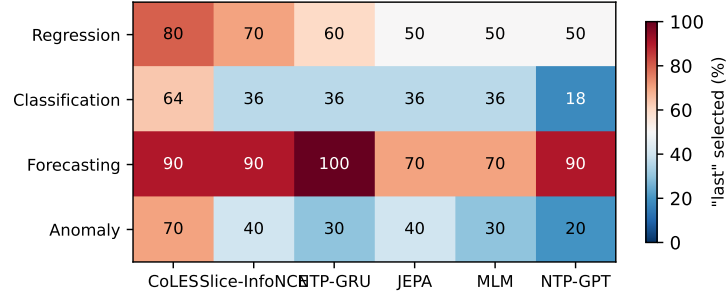


Figure 13: Share of configurations in which the last hidden state is selected over mean-pooling, per task family and method.

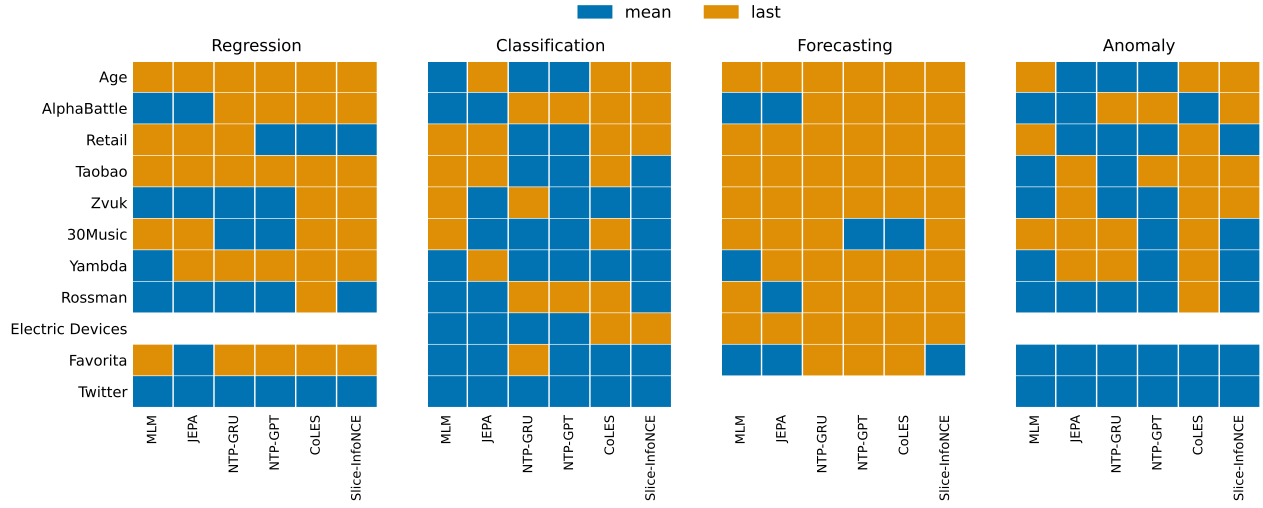


Figure 14: Aggregation selected by hyperparameter optimisation for the dataset-method combinations included in this analysis, per task family. Blue = mean-pooling, orange = last hidden state; empty cells correspond to dataset-task combinations absent from the benchmark.

Table 19: Detailed first-versus-final validator ranking summary. Lower rank and regret are better.

Statistic	Scope	First 10 Optuna trials			Final 10 Optuna trials		
		LightGBM	MLP	Linear probe	LightGBM	MLP	Linear probe
Winner	Total	10	7	6	11	5	7
Rank	Avg.	1.783	1.870	2.261	1.652	2.000	2.043
Rank	Cls.	1.833	1.500	2.500	1.167	2.167	2.500
Rank	Forecast	2.000	2.200	1.800	2.000	1.400	1.800
Rank	Reg.	1.000	2.333	2.667	1.333	2.333	2.333
Rank	Anomaly	2.333	1.500	2.000	2.167	2.000	1.500
Rank-S	Avg.	1.779	1.875	2.021	1.729	1.929	2.079
Regret	Avg.	0.334	1.075	0.665	0.345	0.971	0.840

This approach avoids constructing the large fusion tensor explicitly while producing the same mode-wise projection matrices as the Tucker HOSVD [10] solution.

A More General Problem Setup. Consider two views $\mathbf{x}_1 \in \mathbb{R}^{n_1}$, $\mathbf{x}_2 \in \mathbb{R}^{n_2}$. An MLP layer with weight matrix $\mathbf{W} \in \mathbb{R}^{n_1 \times n_2}$ can be written as

$$f_{\mathbf{W}}(\mathbf{x}_1, \mathbf{x}_2) = \mathbf{x}_1^T \mathbf{W} \mathbf{x}_2 = \text{trace}(\mathbf{W}^T (\mathbf{x}_1 \mathbf{x}_2^T)). \quad (23)$$

Define the rank-1 interaction matrix $\mathbf{X} = \mathbf{x}_1 \mathbf{x}_2^T$; then the model becomes

$$f(\mathbf{x}_1, \mathbf{x}_2) = \text{trace}(\mathbf{W}^T \mathbf{X}). \quad (24)$$

This shows where the rank-1 interaction comes from.

Now replace $\mathbf{W}$ by its low-rank factorization $\mathbf{W} = \mathbf{A} \mathbf{B}^T$, where $\mathbf{A} \in \mathbb{R}^{n_1 \times r}$, $\mathbf{B} \in \mathbb{R}^{n_2 \times r}$. Then

$$f(\mathbf{x}_1, \mathbf{x}_2) = \text{trace}((\mathbf{A} \mathbf{B}^T)^T (\mathbf{x}_1 \mathbf{x}_2^T)) = (\mathbf{A}^T \mathbf{x}_1)^T (\mathbf{B}^T \mathbf{x}_2). \quad (25)$$

Imposing orthogonality constraints on **A** and **B** leads exactly to the Tucker-based fusion problem discussed earlier.

Extension to Multiple Views. For more than two views, the interaction can be modeled as a Tensor Train [33] (TT) tensor

$$\mathcal{X} = \text{TT}(\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_d), \quad (26)$$

and the weight tensor $\mathcal{W}$ can also be represented in TT format. This leads to a natural multi-way generalization of the low-rank fusion framework, which is a direction for future investigation.

E.5 Embedding Fusion Analysis

Tables 6–?? report the full AGE and Rossmann results for all six pairs of task-best encoders under both validators. For the CoLES–JEPa pair, CCA diverged under the linear probe on the two regression-type tasks ($R^2 < -10$); the affected entries are marked with a dash. Overall, TuckerFactorConcat is the dominant fusion strategy on the AGE dataset, outperforming other methods in the majority of task pairs, particularly for forecasting (R^2) and anomaly detection (ROC-AUC). On Rossmann, the single-model baselines are already highly discriminative; consequently, the benefits of feature fusion are smaller, and TuckerFactorConcat, concatenation, and PCA perform similarly, with no single method clearly winning across all tasks. This is to be expected: despite its representational power, feature fusion constitutes a separate optimisation problem – the selection of the rank in the Tucker decomposition is NP-hard – and a learning problem that, like self-supervised learning, depends critically on the quality of the input embeddings; realising its full potential may require more careful hyperparameter tuning than simply fixing the rank to the minimum of the two feature dimensions.

E.6 Single- vs Multi-Objective Selection: Details

This appendix expands the configuration trade-off and HPO comparison of Section 5.5. Tables 32–35 give the full per-configuration results underlying Table 5. Each row is one configuration: *Stop-by* indicates the downstream metric used to select it, and the four value columns report its performance on all tasks. The last row repeats the corresponding full-benchmark configuration for reference.

The best value for a task is not necessarily obtained by the configuration selected *for* that task: the diagonal entry (*Stop-by* equal to the evaluated task) is often not the column maximum. For instance, on X5-Retail with NTP-GRU the best regression score among the multi-target rows comes from the configuration selected by forecasting (0.121), not the one selected by regression (0.118). Since every candidate is validated on every task, we therefore report in Table 5 the best value each regime attains per task (the column maximum), not the diagonal.

These results use a deliberately small sample, two datasets and two encoders, and should be read as an illustration rather than a full study; the reference rows show that the single- and multi-target numbers stay close to the full-benchmark configuration, so the small-sample comparison is not distorted.

Table 32: X5-Retail, NTP-GRU: single- vs multi-target selection.

HPO	Stop-by	Reg.	Clf.	Forec.	Anom.
<i>Single-target HPO</i>					
Single	Reg.	0.160	0.417	0.395	0.668
Single	Clf.	0.097	0.454	0.156	0.658
Single	Forec.	0.060	0.442	0.431	0.650
Single	Anom.	0.082	0.435	0.132	0.683
<i>Multi-target HPO</i>					
Multi	Reg.	0.118	0.339	0.348	0.640
Multi	Clf.	0.103	0.443	0.133	0.658
Multi	Forec.	0.121	0.426	0.428	0.648
Multi	Anom.	0.076	0.407	0.140	0.663
Full benchmark (ref.)		0.138	0.463	0.412	0.674

Table 33: X5-Retail, JEPa: single- vs multi-target selection.

HPO	Stop-by	Reg.	Clf.	Forec.	Anom.
<i>Single-target HPO</i>					
Single	Reg.	0.125	0.305	0.059	0.639
Single	Clf.	-0.053	0.463	0.075	0.673
Single	Forec.	0.044	0.392	0.325	0.651
Single	Anom.	0.070	0.322	0.094	0.691
<i>Multi-target HPO</i>					
Multi	Reg.	0.146	0.355	0.272	0.660
Multi	Clf.	0.063	0.434	0.185	0.591
Multi	Forec.	0.091	0.441	0.336	0.657
Multi	Anom.	0.041	0.427	0.132	0.692
Full benchmark (ref.)		0.062	0.479	0.403	0.667

Table 34: Twitter, NTP-GRU: single- vs multi-target selection.

HPO	Stop-by	Reg.	Clf.	Forec.	Anom.
<i>Single-target HPO</i>					
Single	Reg.	0.685	0.718	–	0.995
Single	Clf.	0.626	0.744	–	0.993
Single	Anom.	0.566	0.700	–	0.997
<i>Multi-target HPO</i>					
Multi	Reg.	0.677	0.729	–	0.995
Multi	Clf.	0.670	0.751	–	0.996
Multi	Anom.	0.666	0.731	–	0.997
Full benchmark (ref.)		0.689	0.748	–	0.998

Table 20: Fusion of CoLES (Regression) and NTP-GRU (Forecasting) on AGE under both validators. Values are mean $\pm$ standard deviation across repeated downstream runs. The best mean per column within each validator block is in bold, the second best is underlined.

		Classification accuracy	Regression R^2	Forecasting R^2	Anomaly ROC-AUC
Linear probe	CoLES	0.597 $\pm$ 0.002	0.511 $\pm$ 0.001	0.385 $\pm$ 0.002	0.777 $\pm$ 0.003
	NTP-GRU	0.602 $\pm$ 0.003	0.491 $\pm$ 0.000	0.411 $\pm$ 0.002	0.673 $\pm$ 0.006
	CCA	0.590 $\pm$ 0.003	0.510 $\pm$ 0.002	0.412 $\pm$ 0.001	0.764 $\pm$ 0.001
	KrossFuse	0.608 $\pm$ 0.001	0.500 $\pm$ 0.001	0.407 $\pm$ 0.002	0.749 $\pm$ 0.006
	PCA	<u>0.617 $\pm$ 0.004</u>	<u>0.512 $\pm$ 0.001</u>	0.418 $\pm$ 0.001	<u>0.785 $\pm$ 0.003</u>
	TuckerFactorConcat concatenation	0.620 $\pm$ 0.002	0.513 $\pm$ 0.002	0.420 $\pm$ 0.000	0.799 $\pm$ 0.005
LightGBM	CoLES	0.599 $\pm$ 0.004	0.520 $\pm$ 0.000	<u>0.409 $\pm$ 0.001</u>	0.771 $\pm$ 0.004
	NTP-GRU	0.597 $\pm$ 0.001	0.469 $\pm$ 0.000	0.405 $\pm$ 0.000	0.654 $\pm$ 0.005
	CCA	0.581 $\pm$ 0.003	0.496 $\pm$ 0.000	0.408 $\pm$ 0.000	0.749 $\pm$ 0.009
	KrossFuse	<u>0.611 $\pm$ 0.003</u>	0.500 $\pm$ 0.000	0.406 $\pm$ 0.001	0.715 $\pm$ 0.003
	PCA	<u>0.604 $\pm$ 0.006</u>	0.492 $\pm$ 0.000	0.400 $\pm$ 0.001	<u>0.779 $\pm$ 0.007</u>
	TuckerFactorConcat concatenation	0.606 $\pm$ 0.007	0.507 $\pm$ 0.000	0.408 $\pm$ 0.000	0.800 $\pm$ 0.006

Table 21: Fusion of Slice-InfoNCE (Classification) and JEPa (Anomaly) on AGE under both validators. Values are mean $\pm$ standard deviation across repeated downstream runs. The best mean per column within each validator block is in bold, the second best is underlined. Dashes mark diverged CCA runs ($R^2 < -10$).

		Classification accuracy	Regression R^2	Forecasting R^2	Anomaly ROC-AUC
Linear probe	Slice-InfoNCE	0.630 $\pm$ 0.000	0.459 $\pm$ 0.002	0.244 $\pm$ 0.001	0.745 $\pm$ 0.002
	JEPa	0.442 $\pm$ 0.007	0.378 $\pm$ 0.003	-0.011 $\pm$ 0.008	<u>0.982 $\pm$ 0.000</u>
	CCA	0.597 $\pm$ 0.006	—	—	0.932 $\pm$ 0.001
	KrossFuse	<u>0.627 $\pm$ 0.001</u>	0.424 $\pm$ 0.001	0.226 $\pm$ 0.004	0.868 $\pm$ 0.002
	PCA	0.623 $\pm$ 0.003	0.460 $\pm$ 0.001	<u>0.230 $\pm$ 0.006</u>	0.982 $\pm$ 0.001
	TuckerFactorConcat concatenation	0.623 $\pm$ 0.002	<u>0.459 $\pm$ 0.001</u>	0.223 $\pm$ 0.002	0.988 $\pm$ 0.000
LightGBM	Slice-InfoNCE	<u>0.627 $\pm$ 0.004</u>	0.472 $\pm$ 0.001	0.399 $\pm$ 0.000	0.758 $\pm$ 0.008
	JEPa	0.486 $\pm$ 0.002	0.376 $\pm$ 0.001	-0.058 $\pm$ 0.003	<u>0.988 $\pm$ 0.000</u>
	CCA	0.591 $\pm$ 0.006	0.419 $\pm$ 0.000	0.102 $\pm$ 0.001	0.946 $\pm$ 0.003
	KrossFuse	0.622 $\pm$ 0.007	0.431 $\pm$ 0.000	0.237 $\pm$ 0.001	0.820 $\pm$ 0.001
	PCA	0.614 $\pm$ 0.002	0.431 $\pm$ 0.000	0.264 $\pm$ 0.000	0.978 $\pm$ 0.001
	TuckerFactorConcat concatenation	0.617 $\pm$ 0.005	0.443 $\pm$ 0.000	0.277 $\pm$ 0.000	0.992 $\pm$ 0.000

Table 35: Twitter, JEPa: single- vs multi-target selection.

HPO	Stop-by	Reg.	Clf.	Forec.	Anom.
<i>Single-target HPO</i>					
Single	Reg.	0.389	0.584	—	0.803
Single	Clf.	0.219	0.601	—	0.929
Single	Anom.	0.450	0.647	—	0.998
<i>Multi-target HPO</i>					
Multi	Reg.	0.480	0.634	—	0.994
Multi	Clf.	0.477	0.646	—	0.991
Multi	Anom.	0.461	0.630	—	0.994
Full benchmark (ref.)		0.498	0.635	—	0.999

F Data Splitting Protocol

Global Time Split. For datasets with local targets, we follow the Global Time Split of Time to Split [20]. We collect all event times that can serve as cutoffs with sufficient future context and define T_{test} as their global 0.9 quantile. This is one timestamp shared by all entities: events before it are available for training and validation, while events at or after it are reserved for testing, regardless of how many events each entity contributes on either side. Each dataset uses a horizon suited to its temporal resolution and activity scale; Table 8 reports the values used to construct local targets and exclude cutoffs without enough future context.

Table 22: Fusion of Slice-InfoNCE (Classification) and NTP-GRU (Forecasting) on AGE under both validators. Values are mean $\pm$ standard deviation across repeated downstream runs. The best mean per column within each validator block is in bold, the second best is underlined.

		Classification accuracy	Regression R^2	Forecasting R^2	Anomaly ROC-AUC
Linear probe	Slice-InfoNCE	0.630 $\pm$ 0.000	0.459 $\pm$ 0.002	0.244 $\pm$ 0.001	<u>0.745 $\pm$ 0.002</u>
	NTP-GRU	0.602 $\pm$ 0.003	0.491 $\pm$ 0.000	0.411 $\pm$ 0.002	0.673 $\pm$ 0.006
	CCA	0.620 $\pm$ 0.003	0.486 $\pm$ 0.000	0.321 $\pm$ 0.001	0.725 $\pm$ 0.005
	KrossFuse	0.621 $\pm$ 0.003	0.454 $\pm$ 0.001	0.358 $\pm$ 0.001	0.685 $\pm$ 0.005
	PCA	0.627 $\pm$ 0.002	0.494 $\pm$ 0.001	0.404 $\pm$ 0.000	0.757 $\pm$ 0.010
	TuckerFactorConcat concatenation	0.631 $\pm$ 0.002 <u>0.630 $\pm$ 0.002</u>	<u>0.497 $\pm$ 0.001</u> 0.498 $\pm$ 0.002	<u>0.411 $\pm$ 0.001</u> 0.411 $\pm$ 0.001	0.729 $\pm$ 0.004 0.741 $\pm$ 0.006
LightGBM	Slice-InfoNCE	<u>0.627 $\pm$ 0.004</u>	0.472 $\pm$ 0.001	0.399 $\pm$ 0.000	<u>0.758 $\pm$ 0.008</u>
	NTP-GRU	0.597 $\pm$ 0.001	0.469 $\pm$ 0.000	<u>0.405 $\pm$ 0.000</u>	0.654 $\pm$ 0.005
	CCA	0.610 $\pm$ 0.002	0.470 $\pm$ 0.000	0.328 $\pm$ 0.000	0.712 $\pm$ 0.009
	KrossFuse	0.623 $\pm$ 0.004	0.442 $\pm$ 0.000	0.371 $\pm$ 0.001	0.684 $\pm$ 0.011
	PCA	0.623 $\pm$ 0.003	0.471 $\pm$ 0.000	0.394 $\pm$ 0.000	0.757 $\pm$ 0.005
	TuckerFactorConcat concatenation	0.624 $\pm$ 0.002 0.629 $\pm$ 0.004	<u>0.481 $\pm$ 0.000</u> 0.488 $\pm$ 0.000	0.403 $\pm$ 0.001 0.422 $\pm$ 0.000	0.718 $\pm$ 0.010 0.760 $\pm$ 0.006

Table 23: Fusion of CoLES (Regression) and Slice-InfoNCE (Classification) on AGE under both validators. Values are mean $\pm$ standard deviation across repeated downstream runs. The best mean per column within each validator block is in bold, the second best is underlined.

		Classification accuracy	Regression R^2	Forecasting R^2	Anomaly ROC-AUC
Linear probe	Slice-InfoNCE	0.630 $\pm$ 0.000	0.459 $\pm$ 0.002	0.244 $\pm$ 0.001	0.745 $\pm$ 0.002
	CoLES	0.597 $\pm$ 0.002	<u>0.511 $\pm$ 0.001</u>	<u>0.385 $\pm$ 0.002</u>	0.777 $\pm$ 0.003
	CCA	0.611 $\pm$ 0.002	0.489 $\pm$ 0.001	0.306 $\pm$ 0.001	<u>0.803 $\pm$ 0.004</u>
	KrossFuse	0.624 $\pm$ 0.007	0.486 $\pm$ 0.001	0.267 $\pm$ 0.000	0.731 $\pm$ 0.005
	PCA	0.624 $\pm$ 0.002	0.503 $\pm$ 0.001	0.339 $\pm$ 0.000	0.789 $\pm$ 0.007
	TuckerFactorConcat concatenation	0.624 $\pm$ 0.001 <u>0.625 $\pm$ 0.003</u>	0.509 $\pm$ 0.001 0.512 $\pm$ 0.001	0.375 $\pm$ 0.001 0.389 $\pm$ 0.001	0.810 $\pm$ 0.002 0.798 $\pm$ 0.004
LightGBM	Slice-InfoNCE	<u>0.627 $\pm$ 0.004</u>	0.472 $\pm$ 0.001	0.399 $\pm$ 0.000	0.758 $\pm$ 0.008
	CoLES	0.599 $\pm$ 0.004	0.520 $\pm$ 0.000	<u>0.409 $\pm$ 0.001</u>	0.771 $\pm$ 0.004
	CCA	0.603 $\pm$ 0.005	0.478 $\pm$ 0.000	0.322 $\pm$ 0.001	0.788 $\pm$ 0.005
	KrossFuse	0.624 $\pm$ 0.004	0.494 $\pm$ 0.000	0.278 $\pm$ 0.000	0.704 $\pm$ 0.004
	PCA	0.615 $\pm$ 0.000	0.493 $\pm$ 0.001	0.311 $\pm$ 0.000	0.773 $\pm$ 0.010
	TuckerFactorConcat concatenation	0.616 $\pm$ 0.003 0.629 $\pm$ 0.004	0.506 $\pm$ 0.000 <u>0.519 $\pm$ 0.001</u>	0.365 $\pm$ 0.001 0.423 $\pm$ 0.000	0.812 $\pm$ 0.008 <u>0.804 $\pm$ 0.007</u>

AlphaBattle source split. AlphaBattle provides only relative within-client times rather than an absolute timeline shared across clients, so a dataset-wide T_{test} cannot be recovered. For its local targets, we therefore preserve the source-provided train and test partitions and sample cutoffs independently within them. Its global targets use the same entity-level split as the other datasets.

Global User Split. Global targets, used mainly for classification and anomaly detection, describe an entire entity rather than the

future after a cutoff. A temporal split is therefore unnecessary. Following the Global User Split of Time to Split [20], we assign complete entity histories to disjoint downstream training and test subsets in an approximately 90/10 proportion. The encoder and downstream model are trained on one set of entities and evaluated on unseen entities, each represented by an embedding of its full history. Datasets without explicit event time, such as Twitter, use this protocol directly. Exact processed counts are reported in Table 11.

Split metadata. The Parquet rows contain the split information required by both target scopes. Temporal train and test directories encode either the global time boundary or, for AlphaBattle, the source-provided partition. The scalar `global_train` flag records

Table 24: Fusion of CoLES (Regression) and JEPA (Anomaly) on AGE under both validators. Values are mean $\pm$ standard deviation across repeated downstream runs. The best mean per column within each validator block is in bold, the second best is underlined. Dashes mark diverged CCA runs ($R^2 < -10$).

		Classification accuracy	Regression R^2	Forecasting R^2	Anomaly ROC-AUC
Linear probe	CoLES	0.597 $\pm$ 0.002	0.511 $\pm$ 0.001	<u>0.385 $\pm$ 0.002</u>	0.777 $\pm$ 0.003
	JEPA	0.442 $\pm$ 0.007	0.378 $\pm$ 0.003	-0.011 $\pm$ 0.008	0.982 $\pm$ 0.000
	CCA	0.581 $\pm$ 0.003	—	—	0.966 $\pm$ 0.003
	KrossFuse	0.592 $\pm$ 0.002	0.501 $\pm$ 0.003	0.361 $\pm$ 0.003	0.974 $\pm$ 0.001
	PCA	0.605 $\pm$ 0.001	0.516 $\pm$ 0.001	0.387 $\pm$ 0.002	<u>0.991 $\pm$ 0.000</u>
	TuckerFactorConcat	<u>0.605 $\pm$ 0.004</u>	<u>0.513 $\pm$ 0.002</u>	0.364 $\pm$ 0.005	0.991 $\pm$ 0.000
	concatenation	0.588 $\pm$ 0.001	0.507 $\pm$ 0.002	0.370 $\pm$ 0.002	0.984 $\pm$ 0.000
LightGBM	CoLES	<u>0.599 $\pm$ 0.004</u>	0.520 $\pm$ 0.000	0.409 $\pm$ 0.001	0.771 $\pm$ 0.004
	JEPA	0.486 $\pm$ 0.002	0.376 $\pm$ 0.001	-0.058 $\pm$ 0.003	<u>0.988 $\pm$ 0.000</u>
	CCA	0.569 $\pm$ 0.002	0.450 $\pm$ 0.001	0.194 $\pm$ 0.001	0.948 $\pm$ 0.002
	KrossFuse	0.587 $\pm$ 0.001	0.505 $\pm$ 0.001	0.307 $\pm$ 0.000	0.976 $\pm$ 0.001
	PCA	0.593 $\pm$ 0.005	0.498 $\pm$ 0.001	0.316 $\pm$ 0.001	0.981 $\pm$ 0.000
	TuckerFactorConcat	0.587 $\pm$ 0.004	0.500 $\pm$ 0.000	0.349 $\pm$ 0.000	0.991 $\pm$ 0.000
	concatenation	0.603 $\pm$ 0.004	<u>0.514 $\pm$ 0.001</u>	<u>0.403 $\pm$ 0.001</u>	0.987 $\pm$ 0.000

Table 25: Fusion of NTP-GRU (Forecasting) and JEPA (Anomaly) on AGE under both validators. Values are mean $\pm$ standard deviation across repeated downstream runs. The best mean per column within each validator block is in bold, the second best is underlined. Dashes mark diverged CCA runs ($R^2 < -10$).

		Classification accuracy	Regression R^2	Forecasting R^2	Anomaly ROC-AUC
Linear probe	JEPA	0.442 $\pm$ 0.007	0.378 $\pm$ 0.003	-0.011 $\pm$ 0.008	0.982 $\pm$ 0.000
	NTP-GRU	0.602 $\pm$ 0.003	0.491 $\pm$ 0.000	<u>0.411 $\pm$ 0.002</u>	0.673 $\pm$ 0.006
	CCA	0.579 $\pm$ 0.009	—	—	0.947 $\pm$ 0.008
	KrossFuse	0.610 $\pm$ 0.002	0.473 $\pm$ 0.001	0.402 $\pm$ 0.001	0.969 $\pm$ 0.001
	PCA	0.616 $\pm$ 0.001	0.499 $\pm$ 0.001	0.412 $\pm$ 0.000	0.991 $\pm$ 0.001
	TuckerFactorConcat	0.616 $\pm$ 0.004	<u>0.497 $\pm$ 0.001</u>	0.411 $\pm$ 0.001	<u>0.990 $\pm$ 0.001</u>
	concatenation	<u>0.614 $\pm$ 0.004</u>	0.492 $\pm$ 0.000	0.410 $\pm$ 0.001	0.985 $\pm$ 0.000
LightGBM	JEPA	0.486 $\pm$ 0.002	0.376 $\pm$ 0.001	-0.058 $\pm$ 0.003	<u>0.988 $\pm$ 0.000</u>
	NTP-GRU	0.597 $\pm$ 0.001	0.469 $\pm$ 0.000	0.405 $\pm$ 0.000	0.654 $\pm$ 0.005
	CCA	0.580 $\pm$ 0.003	0.441 $\pm$ 0.000	0.132 $\pm$ 0.002	0.938 $\pm$ 0.002
	KrossFuse	0.604 $\pm$ 0.001	0.466 $\pm$ 0.000	0.401 $\pm$ 0.000	0.927 $\pm$ 0.001
	PCA	<u>0.605 $\pm$ 0.004</u>	<u>0.470 $\pm$ 0.001</u>	0.397 $\pm$ 0.000	0.980 $\pm$ 0.000
	TuckerFactorConcat	0.602 $\pm$ 0.005	0.468 $\pm$ 0.000	0.397 $\pm$ 0.001	0.991 $\pm$ 0.000
	concatenation	0.610 $\pm$ 0.005	0.476 $\pm$ 0.000	<u>0.404 $\pm$ 0.000</u>	0.987 $\pm$ 0.000

the entity-level assignment used for global targets. For classification targets, entity splits are stratified when necessary to preserve class proportions and keep rare classes represented in each subset. Fixed seeds make both entity assignments and sampled shifts reproducible.

G Downstream Validator Details

All downstream validators operate on frozen embeddings and are trained independently for each dataset-task pair. We do not tune

validator hyperparameters: each validator uses the same fixed configuration across datasets, pretrained encoders, and embedding dimensions. Only the output dimension and learning objective depend on the downstream task; LightGBM additionally enables class balancing for anomaly detection.

Linear probe. The linear probe is implemented as a single affine layer without hidden layers. Classification and anomaly-detection tasks use cross-entropy, whereas regression and forecasting tasks use mean-squared error. The probe is trained on a GPU with Adam, a learning rate of 3×10^{-4} , weight decay 2×10^{-6} , and batch size 512 for at most 200 epochs. We reserve 10% of the training examples for early stopping on the downstream validation metric, using a

Table 26: Fusion of Slice-InfoNCE (Classification) and CoLES (Anomaly) on Rossmann under both validators. Values are mean $\pm$ standard deviation across repeated downstream runs. The best mean per column within each validator block is in bold, the second best is underlined.

		Classification accuracy	Regression R^2	Forecasting R^2	Anomaly ROC-AUC
Linear probe	Slice-InfoNCE	0.729 $\pm$ 0.022	0.951 $\pm$ 0.000	-0.606 $\pm$ 0.000	<u>0.901</u> $\pm$ 0.001
	CoLES	0.688 $\pm$ 0.000	<u>0.945</u> $\pm$ 0.003	<u>0.053</u> $\pm$ 0.000	0.904 $\pm$ 0.001
	CCA	0.170 $\pm$ 0.009	-2.801 $\pm$ 0.065	-2.018 $\pm$ 0.003	0.659 $\pm$ 0.003
	KrossFuse	0.744 $\pm$ 0.014	0.918 $\pm$ 0.002	-0.915 $\pm$ 0.000	0.882 $\pm$ 0.001
	PCA	0.711 $\pm$ 0.021	0.933 $\pm$ 0.001	-2.049 $\pm$ 0.000	0.865 $\pm$ 0.002
	TuckerFactorConcat concatenation	<u>0.735</u> $\pm$ 0.014	0.932 $\pm$ 0.001	-1.775 $\pm$ 0.000	0.868 $\pm$ 0.002
LightGBM	Slice-InfoNCE	0.756 $\pm$ 0.005	<u>0.961</u> $\pm$ 0.000	0.229 $\pm$ 0.002	0.894 $\pm$ 0.002
	CoLES	0.768 $\pm$ 0.009	0.970 $\pm$ 0.001	0.632 $\pm$ 0.010	<u>0.898</u> $\pm$ 0.003
	CCA	0.577 $\pm$ 0.014	0.175 $\pm$ 0.028	-0.042 $\pm$ 0.023	0.703 $\pm$ 0.006
	KrossFuse	<u>0.765</u> $\pm$ 0.021	0.910 $\pm$ 0.003	0.238 $\pm$ 0.004	0.891 $\pm$ 0.003
	PCA	0.726 $\pm$ 0.005	0.944 $\pm$ 0.001	<u>0.248</u> $\pm$ 0.002	0.879 $\pm$ 0.004
	TuckerFactorConcat concatenation	0.726 $\pm$ 0.005	0.956 $\pm$ 0.000	0.247 $\pm$ 0.001	0.891 $\pm$ 0.002
		0.744 $\pm$ 0.005	0.960 $\pm$ 0.000	0.224 $\pm$ 0.003	0.899 $\pm$ 0.002

Table 27: Fusion of Slice-InfoNCE (Classification) and JEPa (Forecasting) on Rossmann under both validators. Values are mean $\pm$ standard deviation across repeated downstream runs. The best mean per column within each validator block is in bold, the second best is underlined.

		Classification accuracy	Regression R^2	Forecasting R^2	Anomaly ROC-AUC
Linear probe	Slice-InfoNCE	0.729 $\pm$ 0.022	<u>0.951</u> $\pm$ 0.000	-0.606 $\pm$ 0.000	<u>0.901</u> $\pm$ 0.001
	JEPa	0.488 $\pm$ 0.065	0.920 $\pm$ 0.001	0.225 $\pm$ 0.000	0.789 $\pm$ 0.015
	CCA	0.223 $\pm$ 0.039	0.651 $\pm$ 0.008	-2.071 $\pm$ 0.001	0.806 $\pm$ 0.007
	KrossFuse	0.750 $\pm$ 0.015	0.937 $\pm$ 0.003	0.052 $\pm$ 0.001	0.886 $\pm$ 0.005
	PCA	0.708 $\pm$ 0.014	0.947 $\pm$ 0.000	-2.041 $\pm$ 0.000	0.882 $\pm$ 0.001
	TuckerFactorConcat concatenation	0.690 $\pm$ 0.111	0.947 $\pm$ 0.000	-1.053 $\pm$ 0.001	0.886 $\pm$ 0.001
LightGBM	Slice-InfoNCE	0.756 $\pm$ 0.005	0.961 $\pm$ 0.000	0.229 $\pm$ 0.002	0.894 $\pm$ 0.002
	JEPa	0.741 $\pm$ 0.024	0.924 $\pm$ 0.000	0.225 $\pm$ 0.001	0.802 $\pm$ 0.003
	CCA	0.658 $\pm$ 0.010	0.529 $\pm$ 0.005	0.138 $\pm$ 0.003	0.834 $\pm$ 0.002
	KrossFuse	<u>0.765</u> $\pm$ 0.010	0.915 $\pm$ 0.001	0.236 $\pm$ 0.001	0.896 $\pm$ 0.001
	PCA	0.747 $\pm$ 0.010	0.946 $\pm$ 0.002	<u>0.242</u> $\pm$ 0.001	0.887 $\pm$ 0.004
	TuckerFactorConcat concatenation	0.768 $\pm$ 0.009	0.953 $\pm$ 0.000	0.242 $\pm$ 0.000	0.871 $\pm$ 0.005
		0.747 $\pm$ 0.005	<u>0.958</u> $\pm$ 0.000	0.230 $\pm$ 0.002	<u>0.894</u> $\pm$ 0.002

patience of 20 epochs; classification splits are stratified whenever possible. Final results are averaged over validator seeds 0, 1, and 2.

MLP. The nonlinear probe has two hidden layers of widths 256 and 128 with ReLU activations and no dropout. It uses the same optimiser, batch size, validation split, and early-stopping procedure as the linear probe, with a maximum of 1,000 epochs.

LightGBM. We use a fixed GBDT configuration with 1,000 trees, learning rate 0.02, maximum depth 12, 50 leaves, minimum 50 observations per leaf, row and feature subsampling rates of 0.75, and L_1 and L_2 regularisation coefficients equal to 1. Class balancing is enabled only for anomaly-detection tasks. The validator-sensitivity analysis uses random state 42 for all three validators.

H Unsupervised Model-Selection Details

The SSL-loss analysis of Section 5.4 compares hyperparameter configurations, whereas the following metrics are evaluated across checkpoints within a training run. Each metric is computed from frozen representations without downstream labels. Because their numerical ranges differ, the epoch-wise plots normalise each trajectory independently; checkpoint selection depends on the location of a curve's maximum rather than its scale.

RankMe. RankMe is the effective rank of the singular-value distribution of a representation matrix. We compute it both on the final sequence embedding and immediately after categorical and

Table 28: Fusion of CoLES (Regression) and Slice-InfoNCE (Classification) on Rossmann under both validators. Values are mean $\pm$ standard deviation across repeated downstream runs. The best mean per column within each validator block is in bold, the second best is underlined.

		Classification accuracy	Regression R^2	Forecasting R^2	Anomaly ROC-AUC
Linear probe	Slice-InfoNCE	0.729 ± 0.022	0.951 ± 0.000	<u>-0.606</u> ± 0.000	0.901 ± 0.001
	CoLES	0.637 ± 0.066	<u>0.972</u> ± 0.001	-1.032 ± 0.000	<u>0.747</u> ± 0.001
	CCA	0.208 ± 0.014	0.573 ± 0.001	-2.062 ± 0.000	0.722 ± 0.012
	KrossFuse	0.583 ± 0.055	0.933 ± 0.001	-1.615 ± 0.000	0.716 ± 0.004
	PCA	0.604 ± 0.054	0.964 ± 0.000	-2.062 ± 0.000	0.638 ± 0.005
	TuckerFactorConcat concatenation	0.601 ± 0.036 <u>0.688</u> ± 0.018	0.974 ± 0.000 0.966 ± 0.000	-1.903 ± 0.000 -0.163 ± 0.000	0.737 ± 0.004 0.713 ± 0.007
LightGBM	Slice-InfoNCE	<u>0.756</u> ± 0.005	0.961 ± 0.000	0.229 ± 0.002	0.894 ± 0.002
	CoLES	0.747 ± 0.010	0.980 ± 0.000	0.845 ± 0.001	<u>0.867</u> ± 0.001
	CCA	0.592 ± 0.010	0.613 ± 0.002	0.500 ± 0.001	0.769 ± 0.002
	KrossFuse	0.717 ± 0.005	0.927 ± 0.001	0.624 ± 0.001	0.643 ± 0.006
	PCA	0.717 ± 0.014	0.867 ± 0.004	0.621 ± 0.003	0.583 ± 0.004
	TuckerFactorConcat concatenation	0.738 ± 0.005 0.759 ± 0.009	<u>0.967</u> ± 0.001 0.966 ± 0.001	0.817 ± 0.002 <u>0.843</u> ± 0.001	0.785 ± 0.002 0.706 ± 0.006

Table 29: Fusion of CoLES (Regression) and CoLES (Anomaly) on Rossmann under both validators. Values are mean $\pm$ standard deviation across repeated downstream runs. The best mean per column within each validator block is in bold, the second best is underlined.

		Classification accuracy	Regression R^2	Forecasting R^2	Anomaly ROC-AUC
Linear probe	CoLES	<u>0.688</u> ± 0.000	0.945 ± 0.003	<u>0.053</u> ± 0.000	0.904 ± 0.001
	CoLES	0.637 ± 0.066	0.972 ± 0.001	-1.032 ± 0.000	0.747 ± 0.001
	CCA	0.262 ± 0.031	0.938 ± 0.001	-2.055 ± 0.002	0.881 ± 0.005
	KrossFuse	0.682 ± 0.019	<u>0.959</u> ± 0.001	-1.059 ± 0.000	0.898 ± 0.001
	PCA	0.676 ± 0.029	0.918 ± 0.002	-2.056 ± 0.000	0.920 ± 0.001
	TuckerFactorConcat concatenation	0.598 ± 0.086 0.702 ± 0.021	0.936 ± 0.001 0.935 ± 0.002	-1.818 ± 0.000 0.097 ± 0.000	<u>0.920</u> ± 0.001 0.912 ± 0.002
LightGBM	CoLES	<u>0.768</u> ± 0.009	0.970 ± 0.001	0.632 ± 0.010	0.898 ± 0.003
	CoLES	0.747 ± 0.010	0.980 ± 0.000	0.845 ± 0.001	0.867 ± 0.001
	CCA	0.637 ± 0.005	0.970 ± 0.000	0.758 ± 0.001	0.874 ± 0.006
	KrossFuse	0.762 ± 0.010	<u>0.975</u> ± 0.000	0.776 ± 0.001	0.904 ± 0.000
	PCA	0.756 ± 0.010	0.956 ± 0.001	0.725 ± 0.001	0.909 ± 0.002
	TuckerFactorConcat concatenation	0.756 ± 0.005 0.771 ± 0.014	0.958 ± 0.000 0.971 ± 0.000	<u>0.818</u> ± 0.002 0.740 ± 0.003	0.904 ± 0.001 <u>0.904</u> ± 0.003

numerical feature encoding, before the sequence backbone; the latter curve is denoted *first layer*.

LiDAR. LiDAR computes a smooth rank from a discriminant covariance constructed using the SSL views as surrogate classes. *REG* in the figures denotes the diagonal regularisation added to the within-class covariance before inversion. We evaluate the displayed regularisation values because the resulting trajectory can be sensitive to this numerical choice.

aSML. Aligned Sliced Mutual Information evaluates nonlinear dependence after projecting the representations. Its displayed parameters are the Gaussian-noise scale σ , projection dimension d , and number of nearest neighbours k used by the mutual-information

estimator. We evaluate several combinations because no parameter choice can be selected from downstream labels in a label-free regime.

Figures 15–18 provide additional trajectories for AGE and Twitter, NTP-GPT and CoLES, and classification, regression, and anomaly detection. Across these examples, changing the metric, representation layer, or metric parameters changes the selected epoch, and the unsupervised optimum does not consistently coincide with the downstream optimum.

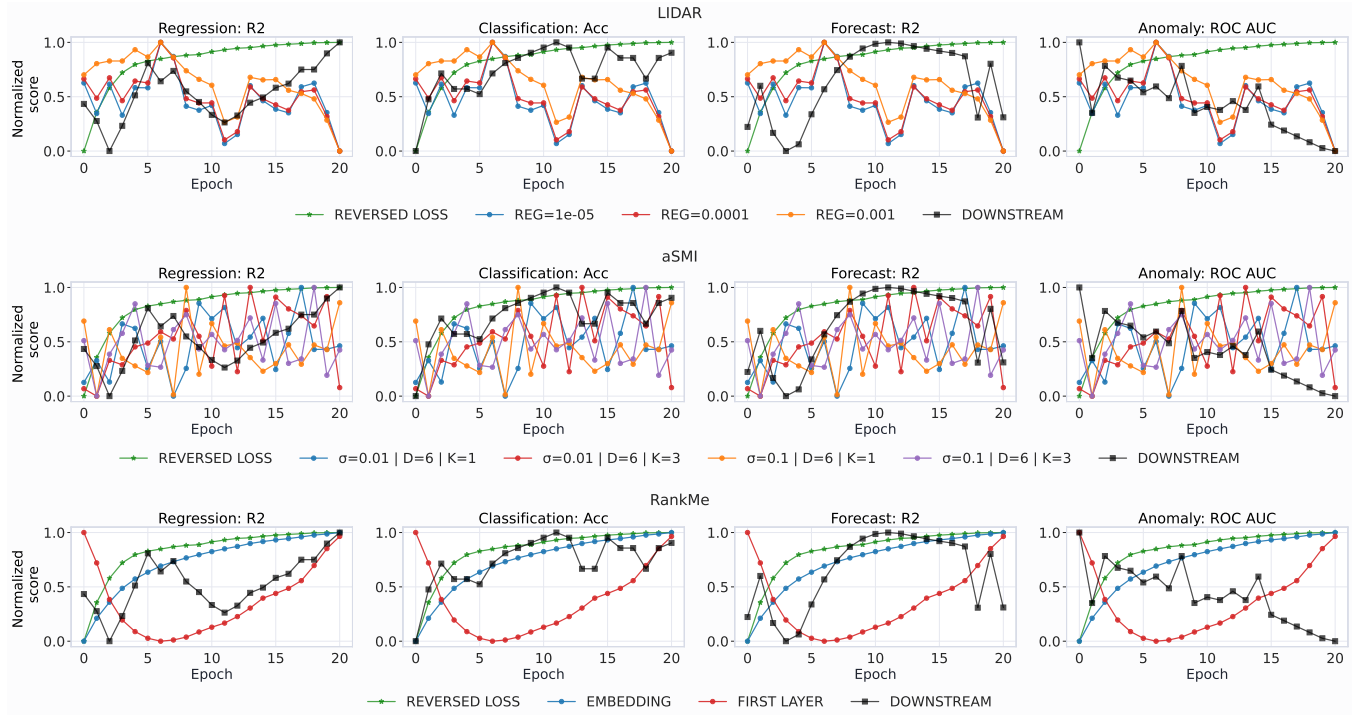


Figure 15: NTP-GPT trained on AGE anomaly detection: epoch-wise RankMe, LiDAR, and aSMI trajectories together with downstream performance and SSL loss.

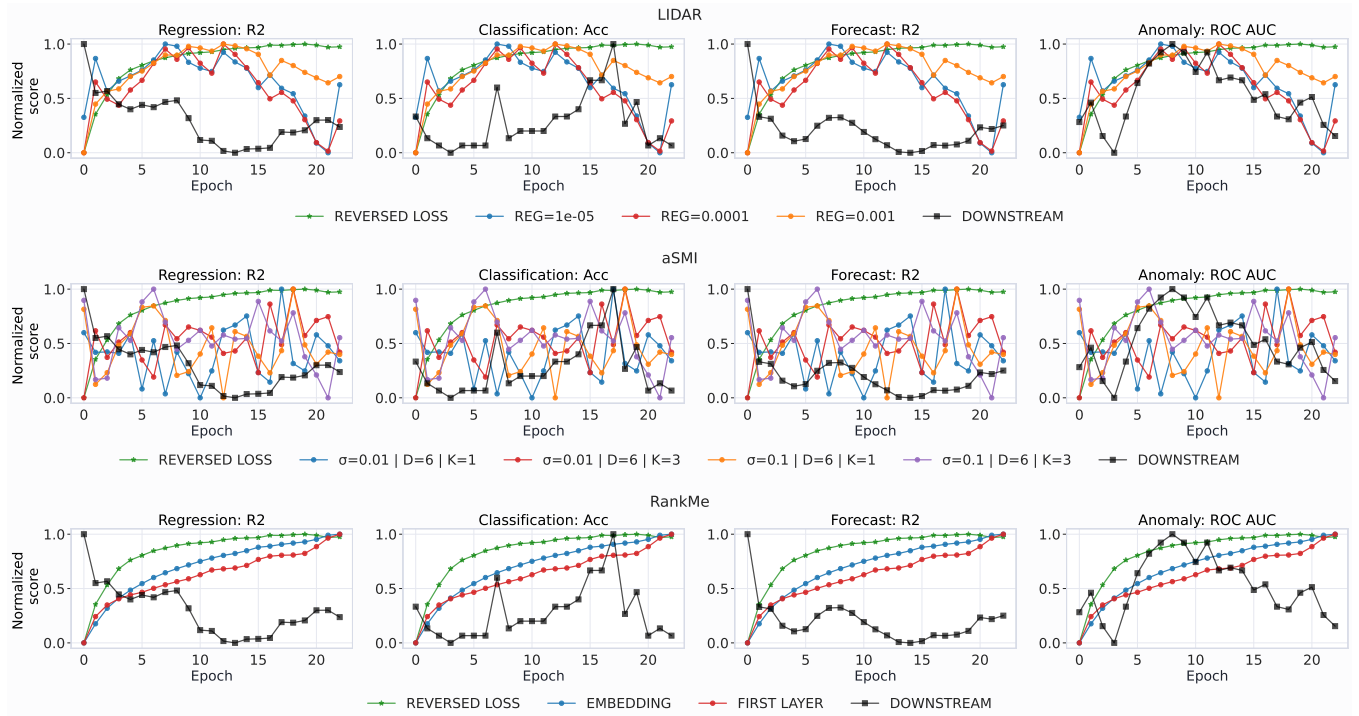


Figure 16: NTP-GPT trained on AGE classification: epoch-wise RankMe, LiDAR, and aSMI trajectories together with downstream performance and SSL loss.

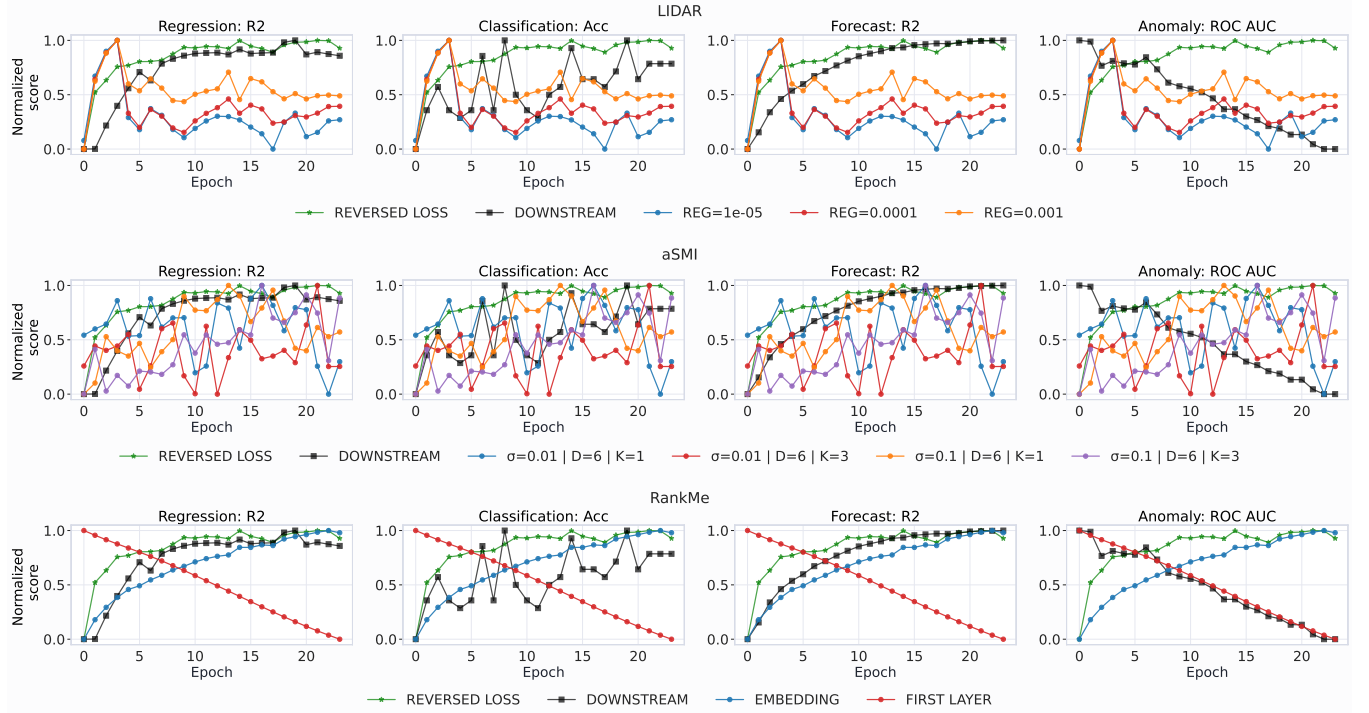


Figure 17: CoLES trained on AGE regression: epoch-wise RankMe, LiDAR, and aSMI trajectories together with downstream performance and SSL loss.

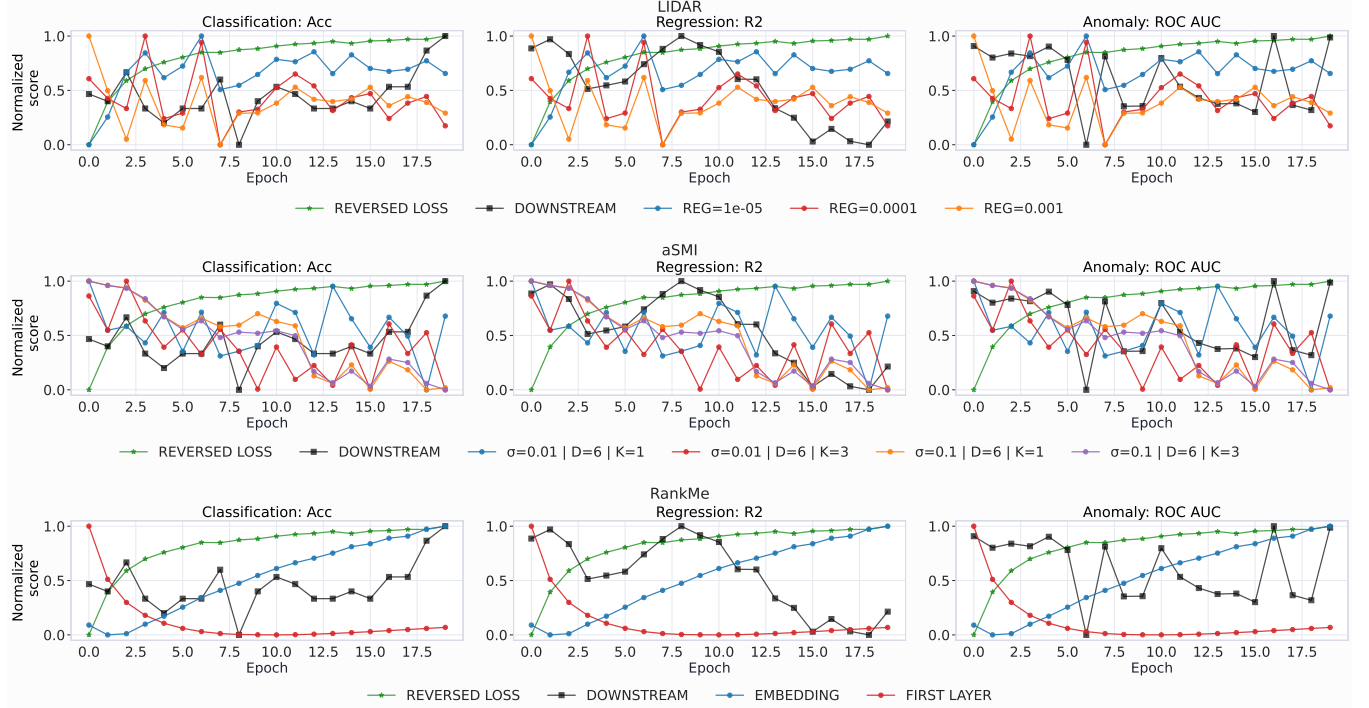


Figure 18: NTP-GPT trained on Twitter classification: epoch-wise RankMe, LiDAR, and aSMI trajectories together with downstream performance and SSL loss.

Table 30: Fusion of CoLES (Regression) and JEPA (Forecasting) on Rossmann under both validators. Values are mean $\pm$ standard deviation across repeated downstream runs. The best mean per column within each validator block is in bold, the second best is underlined.

		Classification accuracy	Regression R^2	Forecasting R^2	Anomaly ROC-AUC
Linear probe	CoLES	0.637 $\pm$ 0.066	0.972 $\pm$ 0.001	-1.032 $\pm$ 0.000	0.747 $\pm$ 0.001
	JEPA	0.488 $\pm$ 0.065	0.920 $\pm$ 0.001	<u>0.225 $\pm$ 0.000</u>	0.789 $\pm$ 0.015
	CCA	0.253 $\pm$ 0.068	0.971 $\pm$ 0.000	-2.057 $\pm$ 0.001	0.882 $\pm$ 0.004
	KrossFuse	0.702 $\pm$ 0.005	0.977 $\pm$ 0.001	0.050 $\pm$ 0.000	0.787 $\pm$ 0.003
	PCA	0.304 $\pm$ 0.082	0.924 $\pm$ 0.001	-1.995 $\pm$ 0.000	0.892 $\pm$ 0.001
	TuckerFactorConcat concatenation	0.571 $\pm$ 0.015 <u>0.693 $\pm$ 0.010</u>	0.951 $\pm$ 0.001 <u>0.974 $\pm$ 0.000</u>	-1.097 $\pm$ 0.001 0.233 $\pm$ 0.000	<u>0.887 $\pm$ 0.001</u> 0.766 $\pm$ 0.002
LightGBM	CoLES	0.747 $\pm$ 0.010	0.980 $\pm$ 0.000	<u>0.845 $\pm$ 0.001</u>	0.867 $\pm$ 0.001
	JEPA	0.741 $\pm$ 0.024	0.924 $\pm$ 0.000	0.225 $\pm$ 0.001	0.802 $\pm$ 0.003
	CCA	0.649 $\pm$ 0.022	<u>0.977 $\pm$ 0.000</u>	0.629 $\pm$ 0.002	0.898 $\pm$ 0.003
	KrossFuse	0.750 $\pm$ 0.009	0.975 $\pm$ 0.000	0.805 $\pm$ 0.001	0.841 $\pm$ 0.002
	PCA	0.774 $\pm$ 0.014	0.944 $\pm$ 0.001	0.776 $\pm$ 0.001	0.874 $\pm$ 0.003
	TuckerFactorConcat concatenation	0.750 $\pm$ 0.009 <u>0.753 $\pm$ 0.005</u>	0.961 $\pm$ 0.000 0.976 $\pm$ 0.000	0.824 $\pm$ 0.001 0.846 $\pm$ 0.001	<u>0.881 $\pm$ 0.004</u> 0.861 $\pm$ 0.002

Table 31: Fusion of JEPA (Forecasting) and CoLES (Anomaly) on Rossmann under both validators. Values are mean $\pm$ standard deviation across repeated downstream runs. The best mean per column within each validator block is in bold, the second best is underlined.

		Classification accuracy	Regression R^2	Forecasting R^2	Anomaly ROC-AUC
Linear probe	CoLES	0.688 $\pm$ 0.000	0.945 $\pm$ 0.003	0.053 $\pm$ 0.000	0.904 $\pm$ 0.001
	JEPA	0.488 $\pm$ 0.065	0.920 $\pm$ 0.001	<u>0.225 $\pm$ 0.000</u>	0.789 $\pm$ 0.015
	CCA	0.244 $\pm$ 0.059	0.954 $\pm$ 0.003	-2.047 $\pm$ 0.002	0.824 $\pm$ 0.026
	KrossFuse	0.723 $\pm$ 0.009	<u>0.952 $\pm$ 0.004</u>	0.241 $\pm$ 0.000	0.906 $\pm$ 0.000
	PCA	0.637 $\pm$ 0.014	0.939 $\pm$ 0.003	-1.989 $\pm$ 0.000	0.917 $\pm$ 0.002
	TuckerFactorConcat concatenation	0.625 $\pm$ 0.101 <u>0.702 $\pm$ 0.027</u>	0.944 $\pm$ 0.002 0.936 $\pm$ 0.002	-0.983 $\pm$ 0.001 0.138 $\pm$ 0.000	<u>0.917 $\pm$ 0.002</u> 0.884 $\pm$ 0.001
LightGBM	CoLES	0.768 $\pm$ 0.009	0.970 $\pm$ 0.001	0.632 $\pm$ 0.010	0.898 $\pm$ 0.003
	JEPA	0.741 $\pm$ 0.024	0.924 $\pm$ 0.000	0.225 $\pm$ 0.001	0.802 $\pm$ 0.003
	CCA	0.565 $\pm$ 0.026	0.757 $\pm$ 0.154	0.406 $\pm$ 0.032	0.872 $\pm$ 0.005
	KrossFuse	<u>0.753 $\pm$ 0.005</u>	<u>0.966 $\pm$ 0.000</u>	0.691 $\pm$ 0.000	0.913 $\pm$ 0.001
	PCA	0.750 $\pm$ 0.018	0.957 $\pm$ 0.000	0.661 $\pm$ 0.003	0.898 $\pm$ 0.003
	TuckerFactorConcat concatenation	0.741 $\pm$ 0.000 0.750 $\pm$ 0.015	0.956 $\pm$ 0.000 0.960 $\pm$ 0.000	<u>0.670 $\pm$ 0.001</u> 0.224 $\pm$ 0.003	0.892 $\pm$ 0.002 <u>0.899 $\pm$ 0.002</u>